Università del Salento
Dipartimento di Fisica e Matematica

Gestione dei dati mancanti in Machine Learning: applicazione a dati clinici per la diagnosi di patologie cardiache

Tesi di Laurea in *Fisica*

Relatore: Prof. Giorgio De Nunzio
Candidato: Lorenzo Rizzo
Matricola: 20028614

Anno Accademico 2024/2025

Indice

1 Introduzione **3**

2 Machine learning e missing data **5**

2.1 Introduzione al machine learning 5

2.1.1 Individuazioni variabili target 6

2.1.2 Acquisizione dati 6

2.1.3 Eventuale preprocessamento dei dati 6

2.1.4 Eventuale bilanciamento del dataset 6

2.1.5 Definizione delle ROI nel caso delle immagini diagnostiche (ma anche di dati in forma di serie temporali) 6

2.1.6 Calcolo delle feature nelle immagini o nelle ROI 7

2.1.7 Riduzione della dimensionalità 8

2.1.8 Scelta di uno schema di addestramento e validazione nel caso dell'addestramento supervisionato 9

2.1.9 Struttura e performance dei classificatori 9

2.2 Problema dei dati mancanti 13

2.2.1 Tipologie di dati mancanti 14

2.3 Strategie di gestione dei dati mancanti 14

2.3.1 Eliminazione dei casi incompleti 14

2.3.2 Tecniche di imputazione semplice 15

2.3.3 Tecniche di imputazione avanzata 16

2.4 k-Nearest Neighbors (KNN) 18

2.5 Regressione multipla 18

2.6 Il metodo MICE 20

2.7 Modelli probabilistici e metodi moderni 22

2.7.1 Autoencoder 23

2.7.2 Altri metodi basati su reti neurali 23

2.8 Metodi “ad albero” per trattare i dati mancanti mediante i “surrogate split” . . . 25
2.8.1 L’algoritmo CART (Classification And Regression Tree) . . . 25
2.8.2 Alberi di Inferenza Condizionale . . . 26
2.8.3 Surrogate Splits . . . 27
2.8.4 Random Forest . . . 28

3 Metodologia **31**
3.1 Il dataset . . . 31
3.1.1 Strumenti utilizzati . . . 32
3.1.2 Metodologia sperimentale . . . 33

4 Risultati **35**
4.1 Risultati . . . 35
4.1.1 Definizione della Baseline . . . 35
4.1.2 Introduzione controllata dei dati mancanti . . . 38
4.1.3 Introduzione del 10% di valori mancanti. Analisi senza e con gli split surrogati . . . 38
4.1.4 Introduzione del 20% di valori mancanti. Analisi senza e con gli split surrogati . . . 43
4.1.5 Confronto complessivo e analisi del degrado delle performance 48

5 Conclusioni **52**

A Codice Sorgente **55**
A.1 Creazione degli alberi decisionali e analisi delle metriche . . . 55
A.2 Generazione dei grafici riassuntivi delle metriche . . . 75

Capitolo 1

Introduzione

L'analisi dei dati sta assumendo un ruolo sempre più centrale nella medicina moderna, trasformando il modo in cui vengono diagnosticate le malattie, personalizzate le terapie e previsti gli esiti clinici. In questo contesto, il **Machine Learning (ML)** offre strumenti potenti per estrarre informazioni preziose da dataset clinici complessi. In particolare, in ambito cardiologico l'applicazione di questi strumenti promette di migliorare significativamente la capacità di individuare precocemente le patologie cardiache, portando a interventi più efficaci e a una migliore gestione del paziente. Il ML, in particolare il tipo supervisionato, ha bisogno di essere "allenato" su un grande insieme di dati, provenienti da un ampio bacino di pazienti, allo scopo di imparare la relazione tra le variabili di ingresso e la variabile clinica da prevedere.

Tuttavia, i dataset medici sono quasi sempre affetti dal problema dei dati mancanti (missing data), dovuti a esami non eseguiti, errori di misurazione o perdite accidentali di informazioni. La maggior parte degli algoritmi di ML, per sua natura, richiede in input dati completi per poter funzionare correttamente. Una gestione inadeguata dei valori mancanti, come la semplice eliminazione dei pazienti con dati incompleti, può portare a una drastica riduzione della numerosità del campione e a modelli predittivi inaffidabili e poco performanti. La gestione dei dati mancanti è quindi un passo critico per qualunque applicazione del Machine Learning in ambito medico. La presente tesi affronta direttamente questa problematica, concentrandosi sulla valutazione di strategie efficaci per la gestione dei dati incompleti nel contesto specifico della diagnosi di patologie cardiache.

Nel dettaglio, la tesi si focalizza sull'analisi di una particolare tecnica dei modelli "ad albero decisionale": gli **split surrogati**. A differenza dei metodi di imputazione (che cercano di assegnare valori attendibili ai dati mancanti), questo meccanismo

permette all'algoritmo decisionale di gestire l'assenza di un dato durante la fase di predizione, sfruttando variabili alternative correlate per prendere una decisione robusta. Per investigare l'efficacia di tale approccio, in questo lavoro di tesi, è stato condotto uno studio sperimentale utilizzando il dataset clinico **UCI Heart Disease**. Il lavoro è partito dalla definizione di un modello di riferimento (baseline) addestrato sul dataset completo. Successivamente, sono state create diverse versioni del dataset introducendo percentuali crescenti di valori mancanti secondo un meccanismo puramente casuale. Su questi dati incompleti sono stati quindi confrontati due approcci: un albero decisionale standard e un albero decisionale che implementa gli split surrogati, al fine di quantificare il degrado delle performance.

La tesi è strutturata come segue. Il **Capitolo 2** introduce i concetti fondamentali del Machine Learning e approfondisce il problema dei dati mancanti, descrivendone le tipologie e le principali strategie di gestione. Il **Capitolo 3** descrive in dettaglio la metodologia sperimentale adottata: vengono presentati il dataset utilizzato e il protocollo che ho seguito per la conduzione degli esperimenti. Il **Capitolo 4** espone e analizza i risultati ottenuti, confrontando le performance dei modelli in termini di accuratezza, sensibilità e specificità al variare della percentuale di dati mancanti. Infine, il **Capitolo 5** riassume le conclusioni del mio lavoro, discutendo le implicazioni dei risultati ottenuti e delineando possibili prospettive e sviluppi futuri.

Capitolo 2

Machine learning e missing data

2.1 Introduzione al machine learning

L'intelligenza artificiale (AI) sta assumendo un ruolo sempre più centrale in medicina, dove permette di integrare una vasta gamma di dati di un paziente, quali stile di vita, microbioma e parametri fisiologici, per delineare un quadro clinico personalizzato. Questo approccio, che consente previsioni con metodi non invasivi, è di fondamentale supporto al medico nelle fasi di diagnosi, prognosi e nella scelta delle terapie.

In questo contesto, un campo di applicazione cruciale è la **Radiomica**, la disciplina che estrae dalle immagini medicali informazioni quantitative non percepibili dall'occhio umano. Gli strumenti software che realizzano questo compito sono noti come sistemi CAD (Computer-Aided Detection/Diagnosis), i quali si distinguono in sistemi **CADe** (Detection), per l'individuazione di potenziali lesioni, e sistemi **CADx** (Diagnosis), per la loro successiva caratterizzazione e classificazione.

Il nucleo di un sistema CAD è un classificatore addestrato a riconoscere schemi ricorrenti, o *pattern*, all'interno dei dati diagnostici. Questo compito rientra nel dominio del **Machine Learning (ML)**, che si articola in due approcci principali: il **ML supervisionato**, in cui un modello apprende da un dataset di esempi già etichettati, e il **ML non supervisionato**, dove è l'algoritmo stesso a scoprire le strutture latenti nei dati senza etichette predefinite.

Un'estensione del problema di classificazione è la regressione, che si concentra sull'individuazione della relazione tra variabili indipendenti e variabili dipendenti. A

differenza della classificazione, in cui la funzione obiettivo assume valori discreti, nella regressione il risultato è una variabile continua, cioè un valore reale.

Lo schema metodologico per l'applicazione del ML in Medicina si compone di diversi step volti ad ottimizzare il processo e ottenere la massima performance.

2.1.1 Individuazioni variabili target

Si individuano le variabili cliniche da utilizzare come *target*. Può essere sia un problema di classificazione se le variabili target sono discrete o un problema di regressione nel caso siano continue.

2.1.2 Acquisizione dati

Si acquisiscono i dati necessari come immagini diagnostiche o serie temporali di variabili cliniche scalari.

2.1.3 Eventuale preprocessamento dei dati

Si processa il dato basandosi su tecniche di trattamento dei dati e algoritmi, per finalità quali la riduzione del rumore o l'evidenziazione dei dettagli nelle immagini.

2.1.4 Eventuale bilanciamento del dataset

Uno dei problemi più importanti che si possono trovare in questa fase sono i dataset sbilanciati. Esiste una multitudine di situazioni e settori in cui una delle classi è molto più rappresentata dell'altra. Ad esempio le lesioni tumorali maligne sono ovviamente minori delle lesioni benigne. A causa di ciò l'algoritmo di ML spesso tende ad inserire anche i casi rari nella classe maggioritaria. Di solito per ovviare al problema si utilizza un approccio di **Data augmentation**. Lo scopo è aumentare la numerosità dei dati della classe minoritaria modificando leggermente i dati già raccolti. Un'alternativa è l'**Undersampling** in cui, al contrario, si riduce la numerosità dei dati appartenenti alla classe maggioritaria.

2.1.5 Definizione delle ROI nel caso delle immagini diagnostiche (ma anche di dati in forma di serie temporali)

Si considerano le immagini diagnostiche raccolte nel dataset. I calcoli delle feature (le caratteristiche in grado di differenziare oggetti appartenenti a classi diverse, vedere il punto successivo) sono operati in regioni di interesse locali (ROI). queste

possono essere lesioni o organi delimitati da un medico o autonomamente con algoritmi di processamento di immagini, oppure possono essere definite con l'approccio a **finestra mobile** dove è attribuita una etichetta a ciascun pixel e quindi il calcolo delle feature è fatto nella finestra stessa. Una volta terminata la fase di etichettatura, l'accorpamento dei pixel equivalenti individua le regioni appartenenti alle diverse classi.

2.1.6 Calcolo delle feature nelle immagini o nelle ROI

A questo punto si passa al calcolo delle *feature*. L'insieme di queste feature può essere visto come un vettore a n componenti. L'insieme dei dati, quindi, si presenta come una matrice con righe pari al numero di campioni e colonne tante quante sono le feature, più la colonna target, che segna l'appartenenza di quel campione ad una delle classi. Le feature possono essere di due tipi: **semantiche** (misure di forma, presenza di margini irregolari ecc.) oppure **agnostiche** (di natura strettamente matematica). Questo processo viene detto **feature extraction**. Di solito, per capire quali siano le feature più adatte da utilizzare, si inseriscono in grandi quantità verificandone a posteriori l'utilità. Si distinguono **feature basate sulla texture** e **feature basate sulla forma**. L'analisi tessiturale si basa sulla valutazione dei livelli di grigio dei pixel in una specifica parte dell'immagine. Esistono diversi tipi di feature basate su tessitura: del *primo ordine*, correlate alla probabilità di osservare un dato livello di grigio in un singolo pixel, di *secondo ordine*, come le *feature di Haralick*, calcolate dalle matrici di co-occorrenza le quali considerano la distribuzione dei grigi su coppie di pixel. Tra le feature basate sulla forma le più importanti sono: la *circolarità*, che descrive quanto una forma possa essere ricondotta ad un cerchio, e la *solidità*, indicatore utile per identificare oggetti con concavità o buchi.

Normalizzazione o standardizzazione delle Feature Per evitare che variabili con range di variazione troppo elevati siano considerate erroneamente più importanti dal classificatore, prima di essere utilizzate, le feature devono essere **normalizzate** tra 0 e 1 oppure **standardizzate** mediante la trasformazione:

$$z_j^k = \frac{x_j^k - \mu_j}{\sigma_j} \tag{2.1}$$

dove μ_j e σ_j sono la media e la deviazione standard della feature x_j dove x_j^k è il valore per il k-esimo campione.

Più feature → più gradi di libertà → più possibilità di "adattarsi" perfettamente ai dati.

2.1.7 Riduzione della dimensionalità

che vuol dire?

L'aumento del numero di feature può portare a problemi di overfitting e a una perdita di qualità. Senza un corrispondente incremento del numero di campioni disponibili, si rischia di incorrere nella cosiddetta *curse of dimensionality*, nella quale l'aumento del numero di feature porta ad una diminuzione della qualità del sistema di **ML**, anziché ad un miglioramento. Per risolvere questo problema è possibile ricorrere a due diverse tecniche per la riduzione della dimensionalità.

Feature selection Mediante questa tecnica si eliminano le variabili in eccesso preservando esclusivamente quelle utili. L'approccio più semplice valuta il potere discriminante delle singole variabili utilizzando, ad esempio, il *coefficiente di Fisher*:

$$F = \frac{(\mu_- - \mu_+)^2}{\sigma_-^2 + \sigma_+^2}, \tag{2.2}$$

dove μ_- e σ_- sono la media e la deviazione standard di una feature per i campioni di una classe mentre μ_+ e σ_+ calcolate su un'altra. Questo coefficiente, quindi, assume valori elevati per feature altamente discriminanti. Questa tecnica, però, non è la migliore dal momento che non considera le relazioni tra feature.

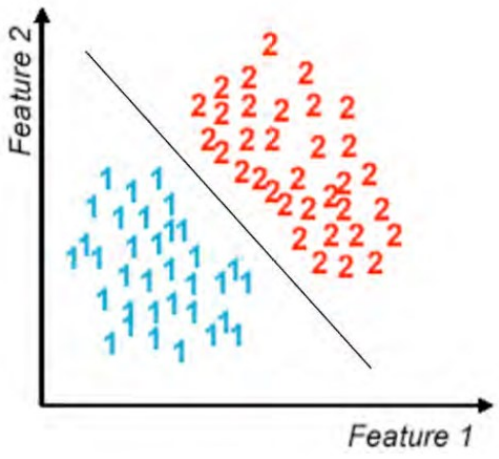


Figura 2.1: Grafico che mostra la relazione tra le feature 1 e 2.

Dalla figura 2.1 si nota che le due feature non sono singolarmente molto discriminanti, tuttavia, considerate insieme, sono perfettamente discriminanti. Allo scopo di tenere conto delle relazioni tra feature, si utilizzano, quindi, tecniche più avanzate come la *Sequential Forward Selection* o la *Sequential Backward Selection*.

Inizi con un modello che non usa nessuna feature. Provi ogni singola feature. Scegli le feature che insieme alle altre danno il risultato migliore.

Inizi con un modello che usa tutte le feature a tua disposizione. Identifichi quale feature, una volta rimossa, ha causato il calo di prestazioni più piccolo e la rimuovi.

la PCA è un metodo non supervisionato che massimizza la varianza globale ignorando le classi, mentre la LDA è supervisionata e massimizza la separazione tra le classi

La PCA fa esattamente questo: trova "l'angolo di visione" (l'asse) migliore per rappresentare i dati.

Feature extraction Questa tecnica applica opportune trasformazioni allo spazio delle variabili con lo scopo di calcolarne di più idonee alla soluzione del problema. Tra queste si citano: la PCA (Principal Component Analysis) e la LDA (Linear Discriminant Analysis).[1][2]

capire cosa sono ?

2.1.8 Scelta di uno schema di addestramento e validazione nel caso dell'addestramento supervisionato

Per quanto riguarda la classificazione supervisionata, sicuramente più diffusa nelle applicazioni alla salute perchè basata sulla verità accertata dai medici in casi precedenti, lo schema di addestramento e validazione prevede la divisione del dataset in diverse parti. Il classificatore inizialmente è allenato sul *training set*, mentre l'apprendimento è verificato sul *validation set*. In questo approccio definito *hold-out cross validation*, il partizionamento è dell'ordine di 70% - 30%. La procedura è iterativa. Modificando leggermente i parametri del modello, ad ogni iterazione, si procede ad allenare il modello per validarlo poi sul validation set. Se la numerosità dei dati lo consente, è importante, oltre al training e validation set, riservare nel dataset un terzo insieme denominato di *test*, adoperato solo per la verifica finale della qualità del sistema allenato. Accanto a questo metodo esistono svariati altri schemi di apprendimento e valutazione. In ambito clinico è importante sottolineare che, al momento del partizionamento dei dati, si dovrà tener in considerazione di non suddividere mai dati appartenenti ad un paziente tra set di allenamento e validazione.

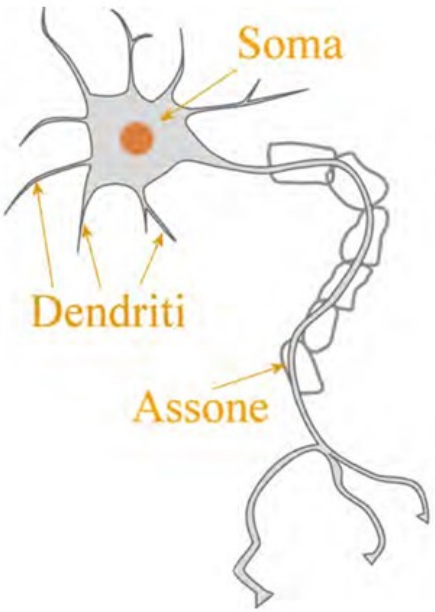
2.1.9 Struttura e performance dei classificatori

Questa sezione spiega come il percettrone (neurone artificiale) possa essere adoperato come classificatore. I neuroni animali sono cellule caratterizzate da tre parti: soma, dendriti e assone. Ognuna di queste tre parti ha una funzione ben precisa. I dendriti sono gli input della cellula: raccolgono i segnali dai neuroni circostanti e li trasmettono al soma, il nucleo centrale. L'assone, invece, è un prolungamento cellulare che costituisce l'output neuronale. Esso ha il compito di trasmettere i segnali prodotti dal neurone ai neuroni circostanti. Fondamentalmente un neurone può essere acceso o spento, ovvero attivo o a riposo. Quando si attiva, il neurone produce uno *spike* elettrico che, raggiunto l'assone e le sue sinapsi (punti di connessione con i dendriti degli altri neuroni), provoca il rilascio di sostanze chimiche (neurotrasmettitori) che penetrano nella cellula collegata per mezzo dei dendriti. Il segnale in arrivo è pesato dalla sinapsi e, se la somma dei segnali pesati in arrivo da ogni assone collegato ai

dendriti del soma supera una certa soglia, allora il neurone si attiva, in caso contrario rimane a riposo. Il cervello umano ospita miliardi di cellule neuronali, ognuna delle quali collegata a centinaia o migliaia di altri neuroni, tutti operanti in parallelo.

Mettiamo ora a confronto la struttura di un neurone animale con un percettrone o neurone artificiale (Figura 2.2), adatto per la realizzazione di una classificazione binaria (a due classi).

Il bias permette di traslare la funzione di attivazione a destra o a sinistra.
Pensa al bias come a una manopola che regola "quanto facile" è per il neurone attivarsi.



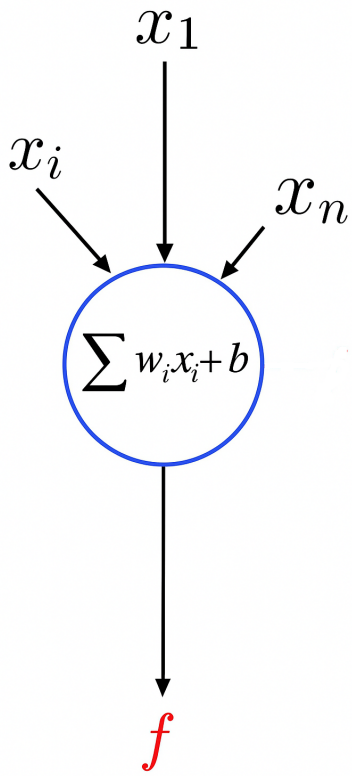


Figura 2.2: A sinistra una cellula neuronale animale, a destra un neurone artificiale.

Notiamo che, come il neurone animale, il percettrone presenta degli ingressi x_i che rappresentano le feature utilizzate dal modello e dei pesi w_i che prendono le veci delle sinapsi e modulano l'intensità di informazione in entrata. Inoltre, sono presenti un input posto a 1 denominato *bias* e una funzione di attivazione $f()$ che dà valori generalmente compresi tra 0 e 1 (come nel caso di una funzione sigmoide, che fornisce uno "score" di appartenenza a una delle due classi) oppure direttamente i valori estremi, classificando il campione in ingresso con l'applicazione di un'operazione di sogliatura. Il valore di output è dato dal risultato di f valutato sulla somma pesata degli ingressi

??

$$y = f(\sum_{i=1} w_i x_i + b) \tag{2.3}$$

Il percettrone è un singolo mattone per la realizzazione di una rete neurale. Esso, valutando dei valori di input, produce un dato output, associato alla classe a cui appartiene il vettore di feature che stiamo considerando. Il neurone artificiale è in grado di apprendere sfruttando la *perceptron learning rule*. Secondo questo metodo si assegnano inizialmente dei pesi randomici e si forniscono come input i vettori del set di training. Ad ogni iterazione i pesi vengono aggiornati secondo la formula:

$$w_{\text{new}} = w_{\text{old}} + \eta(t - o)x_i \tag{2.4}$$

che vuol dire la classe vera

dove x_i è l'i-mo vettore di feature, t e o sono rispettivamente la classe vera alla quale il vettore appartiene e l'output trovato, η è il tasso di apprendimento, un valore utile per controllare la convergenza del processo ($0 < \eta \leq 1$).

Il percettrone singolo ha la capacità di risolvere problemi linearmente separabili, ma per casi più complessi dobbiamo fare riferimento a reti di neuroni. In questi casi la regola di apprendimento sarà in generale più complessa. Le reti neurali prevedono di solito neuroni di input, neuroni di output e strati di neuroni nascosti detti *hidden layers*.

Qualità di un classificatore Una volta addestrato il classificatore sul dataset di training, il sistema sarà idealmente in grado di elaborare un vettore di feature e attribuirlo alla sua classe di appartenenza in maniera accurata. La qualità e le prestazioni del classificatore sono misurate da alcuni parametri come: numero di veri positivi, veri negativi, falsi positivi, falsi negativi, la matrice di confusione, sensibilità, specificità, precisione, curva ROC e curva precision-recall. Con positivi e negativi ci riferiamo, richiamando l'ambito medico, a campioni patologici e sani. Nell'esposizione della struttura del percettrone è stato descritto l'output come determinato dalla media pesata degli input e dalla presenza di una funzione di attivazione che porta usualmente l'output ad un valore continuo tra 0 e 1. Per ottenere un valore discreto, 0 o 1, che rappresenti la classe, è necessario introdurre un criterio di decisione. Questo ruolo, concettualmente analogo a quello svolto dal termine di bias in input, può essere assolto dall'applicazione di una soglia. A tale proposito immaginiamo di applicare una soglia θ_0 sull'output. Nella figura 2.3 i campioni identificati con la lettera a sono effettivamente dei positivi. I campioni identificati con b sono negativi. Alcuni positivi quindi finiranno erroneamente per essere classificati

come negativi (c) e viceversa alcuni negativi verranno classificati come positivi (d). Tenendo conto di ciò, quindi, possiamo definire:

- *sensibilità*: campioni positivi classificati correttamente $TP/(TP+FN)$, TP /(TP+FN)
- *rapporto dei falsi positivi*: $FP/(TN+FP)$,
- *specificità*: campioni negativi classificati correttamente $TN/(TN+FN)$, TN /(TN+FP)
- *accuratezza*: campioni correttamente classificati $(TP+TN)/(TP+TN+FP+FN)$,
- *precisione*: $(TP)/(TP+FP)$,
- *F-measure*: $2\frac{\text{precision}\times\text{sensibilità}}{\text{precision}+\text{sensibilità}}$,
- *Matrice di confusione*: $\begin{pmatrix} TP & FP \\ FN & TN \end{pmatrix}$, TP FN FP TN

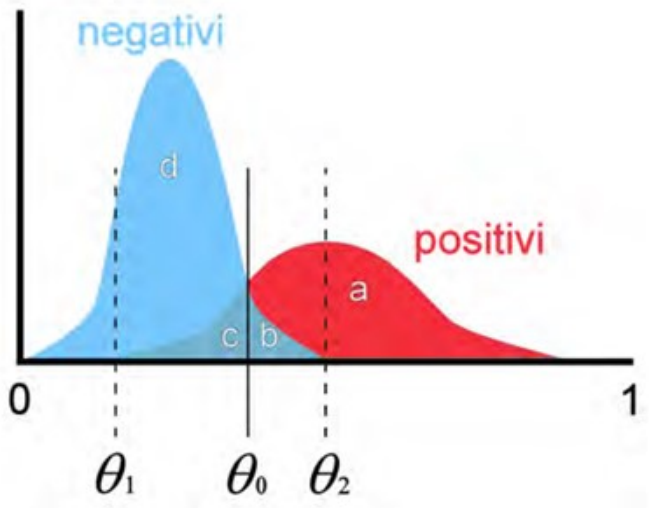


Figura 2.3: La figura illustra le distribuzioni di frequenza per le classi dei positivi (curva rossa) e dei negativi (curva blu) rispetto al punteggio assegnato da un classificatore. La linea verticale continua, posta in corrispondenza di θ_0, rappresenta la soglia di classificazione: i campioni con un punteggio superiore a θ_0 sono etichettati come 'positivi', quelli con un punteggio inferiore come 'negativi'. Questa soglia suddivide lo spazio in quattro aree, che corrispondono ai quattro esiti della matrice di confusione.

Naturalmente, nella situazione ideale si cerca di ottenere una sensibilità alta in modo tale da rivelare, ad esempio tutte le lesioni maligne e alta specificità in modo da

evitare il più possibile falsi allarmi e onerosi conseguenti approfondimenti diagnostici dovuti a lesioni benigne classificate erroneamente.

Machine learning vs Deep learning Il DL è una architettura basata sul ML. Il Deep Learning è nato su ispirazioni dei processi visivi della corteccia visiva cerebrale dei gatti. Questa, infatti, si compone da *cellule semplici* le quali rispondono in maniera intensa ad elementi visivi semplici, di forma lineare, orientati in modo specifico e situati in una parte ben precisa dell'immagine. Altre cellule *complesse* ricevono le informazioni dai neuroni semplici e analizzano l'immagine in maniera più complessa ed estesa, ad un livello superiore. Fukushima fu il primo a realizzare nel 1980 un modello informatico funzionante chiamato *Neocognitron* [3]. In questo modello i primi strati di reti convoluzionali estraggono dalle immagini delle feature di basso livello che poi si combinano fra loro strato per strato. Le feature quindi, sono individuate in maniera autonoma e non supervisionata al contrario del caso del ML. Gli ultimi strati hanno il compito della classificazione finale. Il problema del DL è che una struttura simile necessita di un grandissimo numero di dati (immagini) e quindi, soprattutto in ambito medico, si preferisce spesso utilizzare il ML a causa della scarsità di dati diagnostici.

Tutti i modelli di machine learning descritti, dal semplice percettrone alle complesse reti neurali, condividono un requisito fondamentale: richiedono in input una matrice di feature completa, senza valori mancanti. Nella pratica clinica, tuttavia, i dataset sono quasi sempre incompleti a causa di esami non eseguiti, errori di misurazione o dati persi. Questo pone un problema cruciale: come si possono applicare questi potenti algoritmi quando i dati sono incompleti? La gestione dei dati mancanti non è quindi un problema marginale, ma un passo preliminare e fondamentale per qualsiasi applicazione di successo del machine learning in ambito medico. Le sezioni seguenti affronteranno questo problema in dettaglio, esplorando le diverse tipologie di dati mancanti e le strategie per gestirli.[4]

2.2 Problema dei dati mancanti

Molto spesso, uno dei problemi che caratterizzano i dataset di grandi dimensioni è la presenza di dati mancanti. Le enormi quantità di dati utilizzate quotidianamente rendono, di fatto, la presenza di valori assenti un evento quasi certo. Per questo motivo è necessario implementare metodi in grado di limitare il problema. Se non affrontato, infatti, il gran numero di valori mancanti può comportare una perdita significativa di informazione.

2.2.1 Tipologie di dati mancanti

Prima di poter utilizzare i dati di un dataset, è necessario, quindi, trattare i dati mancanti. Ne esistono di diversi tipi, a seconda di come la loro assenza dipenda dai dati osservati nel dataset. I dati mancanti sono detti **Missing Completely At Random (MCAR)** quando la probabilità che un valore sia assente è completamente indipendente sia dai valori osservati che da quelli non osservati nel dataset.

Un caso più generale è rappresentato dai dati **Missing At Random (MAR)**. In questa situazione, la probabilità che un dato sia mancante può dipendere dai valori già osservati nel dataset, ma non dal valore mancante in sé.

Infine, i dati sono detti **Missing Not At Random (MNAR)** quando la probabilità che un valore sia mancante dipende anche dal valore non osservato. In questo caso, l'assenza del dato è direttamente legata al valore stesso che manca, rendendo la gestione dei dati mancanti più complessa e richiedendo modelli specifici per descrivere il meccanismo di mancanza.

Questa classificazione è necessaria, dal momento che, al variare della tipologia, cambieranno necessariamente le tecniche per trattare i dati mancanti. Purtroppo, tale classificazione è tutt'altro che semplice. Esistono dei metodi statistici per verificare se i dati mancanti sono di tipo MCAR, ma non vi sono tecniche specifiche per distinguere tra MAR e MNAR. L'unico modo è conoscere a priori la natura dei dati raccolti.

2.3 Strategie di gestione dei dati mancanti

2.3.1 Eliminazione dei casi incompleti

Le tecniche più diffuse e utilizzate per trattare i dati mancanti sono quelle che ne prevedono l'eliminazione. Sebbene non siano sempre le più efficaci, sono sicuramente le più conosciute.

Listwise deletion La *Listwise Deletion* è una tecnica molto semplice che consiste nell'eliminare dal dataset tutte le righe che contengono almeno un valore mancante. È sicuramente l'approccio più diffuso in ambito scientifico, ma anche il più distruttivo e potenzialmente pericoloso. Nella maggior parte dei casi, infatti, la perdita di

informazioni è massiccia, soprattutto se il dataset è di piccole dimensioni. Questa tecnica, inoltre, è valida solo nell'ipotesi in cui i dati mancanti siano di tipo MCAR, una condizione piuttosto rara.

Pairwise deletion Un approccio alternativo a quello precedente è la *Pairwise Deletion*. Si tratta di una tecnica indicata per i dataset di modeste dimensioni, che ha il vantaggio di limitare la perdita di informazioni. A differenza del metodo precedente, si considerano di volta in volta solo le variabili di interesse, ovvero quelle direttamente coinvolte nell'analisi, scartando i valori mancanti unicamente per l'analisi in corso. Questo metodo può comportare alcune criticità computazionali, poiché analisi diverse potrebbero basarsi su sottoinsiemi di dati differenti; tuttavia, offre l'importante vantaggio di ridurre la distorsione delle stime rispetto alla Listwise Deletion. ?

2.3.2 Tecniche di imputazione semplice

I metodi di imputazione semplice, pur essendo utili, possono rivelarsi anche pericolosi. Le tecniche adoperate mirano a completare il dataset, laddove si presentino dati mancanti, attraverso l'inserimento di valori stimati ma non realmente osservati. Purtroppo, è proprio questa la principale criticità di tali metodi. Il dataset così ottenuto risulta completo e pronto per le successive fasi di analisi e di training, ma al contempo contiene stime che potrebbero essere distorte, poiché non viene considerata un'adeguata incertezza per i valori inseriti in corrispondenza dei dati mancanti. Le tecniche più usate sono:

- **Media** Si considera la media dei valori assunti da quella specifica variabile nel dataset.
- **Campionamento aleatorio** Il valore mancante viene estratto in maniera del tutto casuale dai valori assunti dalla variabile nel dataset.
- **Regressione** Il valore mancante è dedotto da una regressione tra le variabili osservate.
- **Regressione stocastica** Il valore è dedotto da un processo di regressione a cui è aggiunto un residuo che funge da incertezza.

- **Hot Deck** Sostituisce i dati mancanti con un dato simile, ma realmente osservato.
- **Cold Deck** Sostituisce il dato mancante con un valore della stessa variabile precedentemente osservato in una raccolta di dati precedente.
- **Nearest Neighbor** La tecnica prevede di confrontare le osservazioni utilizzando le variabili presenti, individuando così i nearest neighbors. Il valore mancante viene successivamente imputato in base al valore che la variabile assume per tali osservazioni simili.

2.3.3 Tecniche di imputazione avanzata

L'imputazione multipla è una strategia avanzata che si basa sui metodi di imputazione semplice, ma è progettata per amplificarne i benefici e mitigarne gli aspetti negativi. La caratteristica peculiare che la distingue dall'imputazione semplice è la sua capacità di tenere conto dell'incertezza associata alla generazione dei valori stimati. Fondamentalmente, questo metodo non crea un solo dataset completo, ma ne genera m versioni. In ognuno di questi dataset, i valori mancanti vengono sostituiti utilizzando i metodi di imputazione semplice, ma in modo che ogni stima sia leggermente diversa dalle altre. Una volta ottenuti gli m dataset completi, i risultati delle m analisi vengono combinati per produrre una stima finale unica, che include una misura complessiva dell'incertezza. Come si può intuire, l'aspetto negativo più rilevante è il considerevole dispendio computazionale, poiché il processo di analisi deve essere ripetuto per ognuno dei dataset generati, ma in compenso i vantaggi sono enormi.

Quindi il processo di imputazione multipla può essere diviso di tre fasi diverse:

- **Fase di imputazione** Si generano gli m dataset completi imputando i valori mancanti.
- **Fase di analisi** Si analizzano i dataset con metodi statistici.
- **Fase di pooling** Si unificano le m stime ottenute così da ottenere un unico valore e un'incertezza per il dato mancante.

Per semplicità espositiva si è fatto riferimento al caso di un singolo valore mancante. In generale, tuttavia, l'imputazione multipla viene applicata simultaneamente a tutti i valori mancanti presenti nel dataset: per ciascuna delle m imputazioni viene

generato un dataset completo, in cui ogni dato mancante è sostituito da un valore stimato. Le successive analisi vengono quindi condotte in parallelo sui m dataset così ottenuti.

Fase di imputazione

In questa fase si adottano le tecniche di imputazione singola, combinandole in modo tale che gli m dataset generati risultino leggermente diversi l'uno dall'altro.

Fase di analisi e pooling

Una volta generati gli m dataset completi, si procede alla stima del valore finale del dato mancante. Per ogni dataset $i = 1, \ldots, m$, si calcola il valore del dato mancante θ_i con la relativa incertezza (varianza) U_i per mezzo del modello di imputazione che si è deciso di utilizzare.

Il valore finale θ, ovvero la stima combinata, si ottiene calcolando la media delle singole stime. A questo valore viene poi associata un'incertezza statistica complessiva che tiene conto sia della variabilità all'interno di ciascun dataset sia della variabilità tra i diversi dataset. La stima finale è data da:

ma avevamo detto che non c'era incertezza sui dati generati con imputazione semplice. Quindi come ti spieghi la Ui?

$$\bar{\theta} = \frac{1}{m} \sum_{i=1}^{m} \hat{\theta}_i \tag{2.5}$$

La varianza della stima è:

$$\bar{U} = \frac{1}{m} \sum_{i=1}^{m} \hat{U}_i \tag{2.6}$$

La varianza tra imputazioni B è data da:

$$B = \frac{1}{m-1} \sum_{i=1}^{m} (\hat{\theta}_i - \bar{\theta})^2 \tag{2.7}$$

Da qui la varianza totale associata a θ è:

$$T = \bar{U} + \left(1 + \frac{1}{m}\right) B = \bar{U} + B + \frac{B}{m} \tag{2.8}$$

L'ultimo termine, B/m, rappresenta un fattore di correzione dovuto al fatto che si sta utilizzando un numero finito m di imputazioni. Per un numero m di dataset molto grande, questo termine tende a zero.[5]

2.4 k-Nearest Neighbors (KNN)

Come accennato in precedenza, questa tecnica consiste nel sostituire un dato mancante con il valore proveniente dall'osservazione "più vicina" nel dataset. La procedura si articola in diversi passaggi. Inizialmente, si calcola la distanza tra l'unità con il dato mancante e tutte le altre unità complete, utilizzando un'appropriata funzione di distanza. Successivamente, si identifica l'osservazione più vicina e si utilizza il suo valore per effettuare l'imputazione. Nel caso in cui si utilizzi una sola variabile ausiliaria, la distanza può essere calcolata direttamente sui suoi valori. Tuttavia, quando sono disponibili più variabili, è prassi comune normalizzarle. Questa normalizzazione è essenziale per garantire che tutte le variabili contribuiscano in modo equo al calcolo, evitando che una di esse domini sulle altre a causa di un diverso intervallo di valori. Esistono diverse varianti di questo metodo, che si differenziano per la funzione di distanza utilizzata. Tra queste si hanno: **la distanza Euclidea** e **la distanza Ponderata** (nel caso in cui una variabile sia considerata più importante di un'altra).

Il metodo Nearest Neighbor è particolarmente indicato quando la percentuale di dati mancanti è contenuta. La sua efficacia, invece, diminuisce in dataset con un'alta percentuale di valori mancanti. È inoltre fondamentale sottolineare che il metodo richiede un campione sufficientemente ampio di osservazioni complete per fornire stime affidabili. Quella descritta finora è la procedura per il caso K=1, ma il vero punto di forza del KNN risiede nella possibilità di scegliere un valore di K maggiore di 1. In questo caso, l'imputazione non è una semplice copia, ma viene calcolata aggregando i valori dei K vicini. Per le variabili numeriche, si utilizza la media. L'uso di un K>1 rende il metodo più robusto e meno sensibile a singoli valori anomali.[6]

2.5 Regressione multipla

Regressione semplice Per introdurre il concetto di regressione multipla, è opportuno partire dai modelli più semplici. Un modello di regressione descrive la relazione tra una o più variabili indipendenti X e una variabile dipendente Y. Il caso fondamentale è il modello di regressione lineare semplice, il cui obiettivo è identificare la retta che meglio interpola i dati in un diagramma, descrivendo la relazione

formula dei minimi quadrati??

lineare tra le due variabili. L'equazione di questa retta permette di predire il valore della variabile di output Y a partire da un dato valore della variabile di input X. Il modello assume che la variabile dipendente sia una funzione di quella indipendente più un termine d'errore. Questo errore rappresenta la variabilità non prevedibile di Y. Per stimare i parametri della retta si utilizza il metodo dei minimi quadrati, che identifica l'unica retta che minimizza la somma dei quadrati delle distanze verticali tra i punti osservati e la retta stessa, cioè i residui. Una volta determinata la retta, è possibile calcolare i valori predetti. Per garantire l'affidabilità del modello, devono essere soddisfatte alcune assunzioni fondamentali. Si consideri che nel nostro dataset, per un valore specifico di X, spesso è presente un solo valore osservato di Y. Tuttavia, in linea teorica, se ripetessimo l'esperimento più volte con lo stesso X, otterremmo valori diversi di Y, da cui sarebbe possibile calcolarne media e varianza. Di conseguenza, le assunzioni del modello sono:

1. La media di tutti i valori della variabile Y, per ogni valore specifico della variabile X, giace sulla retta di regressione.
2. La varianza dei valori di Y è costante per tutti i valori di X.
3. Per ogni valore di X, i valori di Y si distribuiscono normalmente attorno alla loro media.
4. Le osservazioni sono indipendenti le une dalle altre.

Infine, qualora la relazione tra le variabili non fosse lineare, si ricorre a modelli di regressione non lineare, che adattano ai dati curve di natura differente.

Regressione multipla Il modello di regressione lineare semplice, descritto nella sezione precedente, costituisce il fondamento per l'analisi delle relazioni tra due variabili. Tuttavia, nella maggior parte dei contesti reali, un fenomeno è raramente influenzato da un'unica variabile. È molto più realistico assumere che Y dipenda simultaneamente da un insieme di più variabili.

A questo punto entra in gioco il concetto di regressione multipla. Questo metodo è simile a quello semplice, ma assume che la variabile di risposta Y non dipenda da una sola variabile predittiva, bensì da un insieme di esse.

Limiti dei metodi classici È fondamentale riconoscere che i metodi di imputazione più semplici, sebbene di facile applicazione, introducono gravi distorsioni nel dataset e sono pertanto considerati inadeguati per la maggior parte delle analisi statistiche. Metodi come la sostituzione con la media o la mediana rappresentano

l'approccio più tradizionale. Il loro difetto principale è duplice: in primo luogo, tendono a ridurre la varianza della variabile e, in secondo luogo, attenuano le relazioni esistenti nel dataset. Un approccio più sofisticato, l'imputazione tramite regressione (sia semplice che multipla), tenta di risolvere il secondo problema. Sfruttando le relazioni tra le variabili, questo metodo fornisce stime più plausibili. Tuttavia introduce un'altra grave distorsione: la sottostima della varianza. Poiché tutti i valori imputati giacciono perfettamente sulla retta di regressione, la variabilità dei dati viene artificialmente ridotta. Per superare i limiti di entrambi gli approcci, si ricorre a metodi avanzati che non solo utilizzano le relazioni tra le variabili, ma tengono anche conto dell'incertezza dei dati mancanti. L'Imputazione Multipla tramite Equazioni Incatenate (MICE) è un esempio emblematico di questa famiglia di tecniche. MICE è eccellente perché: utilizza un modello di regressione multipla per ogni variabile, aggiunge una componente di errore casuale alle predizioni, gestisce la complessità dei dati mancanti presenti su più variabili contemporaneamente attraverso un processo iterativo che migliora le imputazioni fino alla convergenza.[7]

2.6 Il metodo MICE

L'algoritmo MICE è un metodo iterativo: si parte da una prima imputazione dei valori mancanti e si procede attraverso una serie di cicli, dove ad ogni passaggio le stime vengono progressivamente affinate e migliorate. Tecnicamente, MICE è un metodo in cui lo "spazio degli stati" è costituito dall'insieme di tutti i possibili valori imputati. L'affidabilità del metodo si basa sull'assunzione che i dati siano MAR o MCAR. Non è invece indicato per dati mancanti in modo non casuale (MNAR), poiché in tale scenario le imputazioni prodotte sarebbero distorte. Più specificamente, l'algoritmo MICE agisce come un campionatore di Gibbs. Quest'ultimo è un approccio iterativo che opera concentrandosi su una variabile alla volta: mantenendo fisse tutte le altre, "campiona" (cioè sceglie) un valore plausibile per la variabile in esame, basandosi sui valori attuali delle altre. Questo processo viene ripetuto per ogni variabile con dati mancanti in un ciclo continuo. Dopo un numero sufficiente di iterazioni, i valori imputati smettono di cambiare drasticamente e iniziano a stabilizzarsi, raggiungendo la convergenza. In una tipica applicazione del campionatore di Gibbs, le distribuzioni condizionali (la probabilità di una variabile, dato il valore delle altre) sono derivate matematicamente da una distribuzione congiunta di probabilità (la distribuzione che descrive la relazione tra tutte le variabili simultaneamente). In questo scenario classico, la distribuzione congiunta è l'obiettivo teorico a cui l'algoritmo tende a convergere.

?

Nell'algoritmo MICE, invece, la distribuzione congiunta è conosciuta solo implicitamente e potrebbe non esistere affatto. Una catena di Markov, per convergere ad una distribuzione stazionaria, deve soddisfare dei requisiti:

- **Ergodicità** La catena deve essere in grado di esplorare l'intero spazio delle configurazioni plausibili, in modo che ogni regione dello spazio possa essere visitata almeno una volta.
- **Aperiodicità** Il periodo di ritorno nello stesso stato deve essere finito e non dipendere dal numero di cambiamenti di stato.
- **Assorbimento** Deve esistere almeno uno stato che, una volta raggiunto, non può essere abbandonato. Questi sono i cosiddetti "punti di non ritorno".

Si procede esponendo i vari passaggi per l'applicazione del metodo MICE:

- Si specifica un modello di imputazione, come ad esempio una regressione lineare, per ciascuna variabile Y_j con dati mancanti (per $j = 1, \ldots, p$). Tale modello è rappresentato dalla distribuzione condizionale $P(Y_{\text{misj}}|Y_{\text{obsj}}, Y_{-j})$. In altri termini, per stimare i valori mancanti di una variabile (Y_{misj}), si utilizza un modello che si basa sia sui valori osservati della stessa variabile (Y_{obsj}) sia sull'insieme di tutte le altre variabili presenti nel dataset (Y_{-j}).
- Si considera il metodo precedentemente visto chiamato Hot Deck e si riempiono i dati mancanti iniziali con estrazioni casuali tra quelli osservati per la specifica variabile.
- Si itera il processo di imputazione T volte.
- Si ripete il processo per le j variabili.
- Viene definito un set di dati temporaneo tramite la formula:

$$\hat{Y}_{t-j} = \left(\hat{Y}_{t-1,1}, \ldots, \hat{Y}_{t-1,j-1}, \hat{Y}_{t-1,j+1}, \ldots, \hat{Y}_{t-1,p}\right)$$

  La notazione $\hat{Y}_{t-j}$ indica l'insieme di tutte le variabili del dataset (già complete dei valori imputati), ad eccezione della variabile j che è momentaneamente esclusa.

  La parte destra della formula specifica quali dati utilizzare: per l'iterazione corrente t, si prendono i valori che le variabili avevano alla fine dell'iterazione precedente, $t - 1$. In sostanza, si sta considerando lo stato dell'intero dataset al passo precedente, togliendo la colonna j-esima che si andrà a ri-calcolare.

- In questo passaggio ci si concentra a trovare i coefficienti θ adatti al proprio modello di regressione. L'operazione è descritta formalmente dalla seguente estrazione da una distribuzione di probabilità:

$$\hat{\theta}_{tj} \sim P(\theta_{tj}|Y_{\text{obs}j}, \hat{Y}_{t-j})$$

  Questo calcolo viene effettuato applicando il modello ai dati e considerando due elementi fondamentali: i valori osservati della variabile j (cioè $Y_{\text{obs}j}$) e i valori di tutte le altre variabili del dataset (cioè $\hat{Y}_{t-j}$).

- Si utilizzano i coefficienti $\hat{\theta}_{tj}$, estratti al punto precedente, per generare i nuovi dati da imputare nella variabile Y_j. Formalmente, questa operazione è descritta dalla seguente estrazione:

$$\hat{Y}_{tj} \sim P(Y_{\text{mis}j}|Y_{\text{obs}j}, \hat{Y}_{t-j}, \hat{\theta}_{tj})$$

  Il risultato, $\hat{Y}_{tj}$, sarà quindi un nuovo set di valori per i dati mancanti della variabile j, pronti per essere utilizzati nella successiva iterazione dell'algoritmo.

- Questo punto conclude il ciclo per j. Significa che, per l'iterazione t in corso, l'algoritmo ha finito di aggiornare una dopo l'altra tutte le variabili del dataset.

- A questo punto si conclude il ciclo per t. Una volta completato il "giro" su tutte le variabili, si controlla se è stato raggiunto il numero totale di iterazioni T. In caso contrario, se ne inizia una nuova per migliorare ulteriormente i risultati; altrimenti, l'algoritmo si ferma.

Infine, è importante sottolineare che, essendo un metodo di tipo Catena di Markov Monte Carlo, l'algoritmo MICE può presentare potenziali problemi, come la periodicità (in cui l'algoritmo oscilla tra poche soluzioni) o la non ricorrenza (in cui si blocca), che vengono evitati proprio grazie alla progettazione dell'algoritmo. [8]

2.7 Modelli probabilistici e metodi moderni

Accanto ai metodi statistici come MICE, l'apprendimento profondo (Deep Learning) offre architetture innovative per affrontare sia la riduzione della dimensionalità sia il problema dei dati mancanti. Tra queste, un ruolo di primo piano è ricoperto dagli **autoencoder**.

2.7.1 Autoencoder

Un **autoencoder** è una rete neurale addestrata con l'obiettivo di replicare il proprio input nel suo output. La sua architettura si articola in due componenti principali: un **codificatore** (encoder), descritto dalla funzione $h = f(x)$, e un **decodificatore** (decoder), rappresentato dalla funzione $r = g(h)$. Il codificatore mappa l'input x in una rappresentazione interna h, che si trova in uno strato nascosto. Successivamente, il decodificatore utilizza questo "codice" per ricostruire l'input originale.

L'utilità fondamentale di un autoencoder non risiede nella mera copia dell'input, bensì nella sua capacità di apprendere ed estrarre rappresentazioni significative dai dati durante questo processo.

Una strategia comune per raggiungere questo scopo consiste nel vincolare la dimensione del codice h affinché sia inferiore a quella dell'input x. Un'architettura di questo tipo prende il nome di autoencoder **sottocompleto** (undercomplete). Questa restrizione dimensionale costringe la rete a catturare le caratteristiche più salienti e rappresentative dei dati.

Il processo di addestramento è guidato dalla minimizzazione di una funzione di perdita, che penalizza la dissomiglianza tra l'output ricostruito $g(f(x))$ e l'input originale x:

$$L(x, g(f(x)))$$

Le funzioni di codifica f e decodifica g possono essere sia lineari che non lineari. [9]

È fondamentale sottolineare che i metodi statistici tradizionali precedentemente discussi (regressione lineare e non, KNN, MICE, ecc.) presentano limiti significativi, in particolare nel non tenere conto delle complesse dipendenze temporali e non lineari tipiche delle serie di dati. In questo scenario, gli **autoencoder (AE)** emergono come una valida alternativa. Essendo modelli di apprendimento auto-supervisionato, sono in grado di imputare i valori mancanti sfruttando la loro capacità di generare campioni coerenti con i dati di input. [10]

2.7.2 Altri metodi basati su reti neurali

Tra gli altri metodi moderni di imputazione troviamo le reti RNN.

RNN Le **reti neurali ricorrenti (RNN)** sono una classe di reti neurali progettata specificamente per l'elaborazione di dati sequenziali. Queste reti sono specializzate nell'analizzare sequenze di dati, come $x^{(1)}, \ldots, x^{(t)}$, dove l'indice t rappresenta

il passo temporale. Una delle loro caratteristiche distintive è la capacità di processare sequenze di lunghezza variabile, spesso più lunghe di quelle gestibili da reti non specializzate.

Il concetto fondamentale e potente alla base delle RNN è la **condivisione dei parametri** attraverso i diversi passi temporali del modello. Questo meccanismo permette alla rete di mantenere una sorta di "memoria" degli input precedenti. In pratica, lo stato nascosto a un dato tempo t è una funzione dello stato nascosto al tempo precedente $(t-1)$ e dell'input corrente al tempo t.

Per definire i valori delle sue unità nascoste, una rete ricorrente utilizza la seguente equazione di ricorrenza:

$$h(t) = f(h^{(t-1)}, x^{(t)}; \theta)$$

Quando la rete è addestrata per compiti di previsione (predire il futuro basandosi sul passato), lo stato nascosto $h(t)$ impara a funzionare come una **sintesi con perdita di informazione** (in inglese, *lossy summary*) della sequenza di input passata. Questa sintesi cattura gli aspetti più rilevanti per il compito da svolgere.

A seconda dell'obiettivo di addestramento, questa sintesi può conservare selettivamente alcuni aspetti della sequenza passata con maggiore precisione di altri. Ad esempio, in un compito di modellazione del linguaggio (come prevedere la parola successiva date quelle precedenti), la rete potrebbe non aver bisogno di memorizzare l'intera sequenza di input. Sarà sufficiente che conservi le informazioni necessarie a predire il resto della frase in modo coerente.

Come per altre reti neurali, anche le RNN operano tipicamente su **minibatch** (sottoinsiemi del dataset) durante l'addestramento, anziché sull'intero dataset in una sola volta.

Le RNN tradizionali hanno una memoria a brevissimo termine: diventano incapaci di collegare eventi accaduti all'inizio di una lunga sequenza con quelli che accadono alla fine. Per superare questo limite, sono state sviluppate architetture ricorrenti più avanzate, come le LSTM e le GRU, in grado di catturare dipendenze a lungo termine. [11]

E' importante analizzare anche un'altra classe di modelli, i metodi ad albero, che offrono un approccio intrinsecamente diverso alla gestione dei dati mancanti attraverso meccanismi come i "surrogate split".

2.8 Metodi "ad albero" per trattare i dati mancanti mediante i "surrogate split"

I metodi ad albero sono di particolare interesse in quanto sfruttano la logica del partizionamento ricorsivo, un processo che opera tramite suddivisioni binarie e sequenziali dei dati per ottenere sottoinsiemi via via più omogenei. In sostanza, un albero decisionale classifica le osservazioni rispondendo a una serie di domande basate sui valori delle loro singole variabili. Quando si deve fare una previsione, ogni nuova osservazione viene "fatta scendere" lungo l'albero (già creato nella fase di addestramento), seguendo il percorso determinato dalle regole di suddivisione definite in ogni nodo, fino a raggiungere un nodo terminale che fornirà l'esito finale.

2.8.1 L'algoritmo CART (Classification And Regression Tree)

Per determinare le suddivisioni di un albero decisionale vengono generalmente utilizzati due criteri, a seconda del tipo di risposta: l'Indice di Gini per risposte discrete (classificazione) e la Somma dei Quadrati dei Residui (RSS) per risposte continue (regressione). Nel primo caso, l'indice per un dato nodo k è definito come:

$$\hat{G}k = 2\frac{N_{1k}N_{2k}}{N_k N_k} \tag{2.9}$$

dove 1 e 2 indicano le classi di risposta e N il numero totale di osservazioni. L'Indice di Gini è una misura dell'impurità del nodo: assume valore 0 se il nodo è puro (contiene osservazioni di una sola classe) e valore 0.5 se è massimamente impuro. La suddivisione ottimale è quella che massimizza il guadagno di Gini, definito come la differenza tra l'indice del nodo genitore e la somma ponderata degli indici dei nodi figli (sinistro L e destro R):

Non stai portando il Gini verso 0.5.

Stai:

massimizzando la riduzione

quindi minimizzando l'impurità nei nodi figli

quindi portando il Gini verso 0

$$\Delta\hat{G}k = \hat{G}k - \left(\frac{N_{Lk}}{N_k}\hat{G}_{Lk} + \frac{N_{Rk}}{N_k}\hat{G}_{Rk}\right) \tag{2.10}$$

Nel caso di risposte continue, invece, il criterio si basa sulla massimizzazione della riduzione della Somma dei Quadrati dei Residui (RSS):

$$\Delta\widehat{R}_k = \widehat{\mathrm{RSS}}_k - (\widehat{\mathrm{RSS}}_{Lk} + \widehat{\mathrm{RSS}}_{Rk}) \tag{2.11}$$

Una volta stabilito il criterio di crescita, gli alberi vengono sviluppati. Qui può insorgere il problema dell'overfitting: un albero a cui viene permesso di crescere senza limiti imparerà a memoria i dati di addestramento, creando regole troppo specifi-

Divido il training set in K parti (es. 5 o 10).

Per ogni fold:

alleno l'albero grande su K-1 parti

genero la sequenza di alberi potati

valuto l'errore sull'ultima parte

Calcolo l'errore medio per ogni dimensione dell'albero.

che che funzioneranno male su dati nuovi. Per ovviare a ciò, l'albero può essere "potato" (pruned), tagliando i rami meno importanti che aggiungono complessità senza un reale beneficio. La scelta del livello di potatura ottimale avviene tramite validazione incrociata (cross-validation), un metodo che stima le performance del modello su dati inediti. Infine, si seleziona l'albero più semplice la cui performance, stimata tramite validazione incrociata, risulti comparabile a quella dell'albero completamente cresciuto con errore minimo, così da ridurre il rischio di overfitting.

2.8.2 Alberi di Inferenza Condizionale

L'algoritmo CART presenta il difetto di favorire le variabili continue, quelle con molte categorie distinte o con numerosi valori mancanti. Per ovviare a questo problema, si utilizzano gli **Alberi di Inferenza Condizionale**, un metodo che si articola in due passaggi. In una prima fase, viene valutata la relazione tra ciascuna variabile e la risposta tramite un test di permutazione, dal quale si ricava un p-value. La variabile che ottiene il p-value più basso (indice di una relazione più forte) viene quindi selezionata per la suddivisione. Successivamente, nella seconda fase, si determina il punto di taglio ottimale (cutpoint) per la variabile prescelta. Il processo di crescita dell'albero si arresta quando non vengono più individuate relazioni significative.

Metodologia La metodologia si articola in due passaggi fondamentali. In primo luogo, per ogni variabile X_j si valuta l'esistenza di una correlazione con la variabile Y. A tale scopo, si considera la seguente ipotesi nulla H_0:

$$H_0^j : D(Y|X_j) = D(Y) \tag{2.12}$$

che ci dice che le due variabili non sono correlate. H_0 può essere rifiutata se il p-value corrispondente scende al di sotto di un certo livello di significatività.

Utilizziamo una statistica lineare per la correlazione tra X_j e Y:

$$\mathbf{T}_j(\mathcal{L}_n, \mathbf{w}) = \text{vec}\left(\sum_{i=1}^{n} w_i g_j(X_{ji}) h(Y_i, (Y_1, \ldots, Y_n))^\top\right) \in \mathbb{R}^{p_j q} \tag{2.13}$$

Nella parte sinistra dell'equazione:

- $\mathcal{L}_n$ rappresenta il dataset di n osservazioni.
- $\mathbf{w}$ è un vettore di pesi $(w_1, w_2, ..., w_n)$. I pesi permettono di dare più o meno importanza a ciascuna osservazione nel calcolo.

Nella parte destra dell'equazione:

- w_i: È il peso associato alla i-esima osservazione.
- $g_j(X_{ji})$:
  - X_{ji} è il valore della variabile j per l'osservazione i.
  - $g_j(\dots)$ è una **funzione di trasformazione** applicata a X_{ji}. Questa funzione permette di catturare relazioni non lineari. Potrebbe essere una funzione semplice come $g_j(x) = x$ (per relazioni lineari) o qualcosa di più complesso come $g_j(x) = (x, x^2, x^3)$ (per relazioni polinomiali). Il risultato di questa funzione è un vettore di dimensione p_j.
- $h(Y_i, (Y_1, \dots, Y_n))$:
  - Y_i è il valore della variabile di risposta per l'osservazione i.
  - $h(\dots)$ è un'altra **funzione di trasformazione**, questa volta applicata alla variabile di risposta Y.

Quindi per trovare la variabile più importante, utilizziamo un approccio statistico in due fasi. Primo, per ogni variabile calcoliamo una statistica test Tj per misurarne l'associazione con la risposta e ne deriviamo un p-value. Secondo, selezioniamo la variabile con il p-value più basso, cioè quella con l'associazione più forte. Infine, usiamo nuovamente la statistica Tj per identificare il punto di divisione ottimale su quella variabile, ovvero il taglio che separa i dati nel modo più netto. [12]

2.8.3 Surrogate Splits

A questo punto, possiamo distinguere tre approcci per la gestione dei dati mancanti in un albero decisionale.

- **Bloccare l'analisi.** Il primo approccio consiste nel bloccare il percorso di classificazione quando si incontra un valore mancante. In questo caso, la decisione viene presa sulla base delle informazioni disponibili fino a quel nodo, con una conseguente perdita di informazione.
- **Seguire la maggioranza.**La seconda strategia consiste nel far proseguire l'osservazione lungo il ramo più frequentemente seguito in quel nodo. Sebbene semplice, questo metodo permette di continuare l'analisi senza scartare le informazioni successive.

- **Surrogate Splits.** Infine, il terzo e più interessante metodo è quello dei *Surrogate Splits*. Quando un valore è mancante, si ricorre a variabili surrogate. Queste vengono scelte in base al loro livello di correlazione con la variabile mancante, ovvero in base a quanto bene la loro regola di divisione riesca a replicare la scelta tra destra e sinistra della variabile principale. Durante l'addestramento, l'algoritmo non si limita a trovare un solo surrogato, ma crea un'intera classifica basata su questo livello di accordo. In fase di predizione, se anche il valore del miglior surrogato dovesse mancare, il sistema passerà a controllare il secondo in classifica, e così via, fino a trovare una variabile disponibile per poter prendere la decisione.

2.8.4 Random Forest

Un problema noto degli alberi decisionali è la loro instabilità: basta modificare leggermente il dataset di addestramento per ottenere risultati anche molto diversi. Questo crea un'imprevedibilità del metodo che deve essere risolta.

Per ovviare al problema, si ricorre alla tecnica del **bagging**. Con questo approccio, invece di usare un singolo albero, se ne addestra un intero insieme (una "foresta") su vari sottocampioni del dataset principale. La previsione finale è data dalla media o dal voto di maggioranza di tutti gli alberi, portando a una diminuzione sostanziale dell'errore.

Si è notato, infatti, che mentre aumentare la complessità di un singolo albero può portare a problemi, aumentare il numero di alberi nella foresta aiuta quasi sempre a diminuire la varianza. Da qui nascono le **Random Forest**.

L'accuratezza di questo modello viene valutata sulle osservazioni cosiddette **out-of-bag (OOB)**. Durante la creazione dei sottocampioni, alcuni dati del dataset "madre" non vengono mai "pescati". Proprio questi dati, che il modello non ha mai visto in fase di addestramento, vengono poi utilizzati per la validazione.

Misure di Importanza delle Variabili

Le Random Forest non sono impiegate unicamente per la riduzione della varianza, ma anche per l'identificazione delle variabili più rilevanti all'interno di un dataset. Le misure di importanza sono strumenti che permettono di quantificare l'influenza di ciascuna variabile sul modello di Random Forest. Oltre al classico metodo basato sul "bagging", esistono approcci più recenti per questo scopo:

Conteggio delle Suddivisioni Questo metodo determina l'importanza di una variabile contando il numero di volte in cui essa viene utilizzata per effettuare una suddivisione nei diversi alberi della foresta casuale. Sebbene sia di semplice implementazione, il limite di questo approccio risiede nel fatto che non considera né la posizione della suddivisione all'interno dell'albero né il suo effettivo potere discriminatorio. Una variabile che compare in molte suddivisioni a rami profondi, infatti, potrebbe avere un impatto minore rispetto a una che compare in poche ma significative suddivisioni iniziali.

Importanza di Gini L'importanza di Gini misura il contributo di una variabile valutando quanto essa riduca l'impurità dei nodi in cui viene utilizzata per effettuare una suddivisione, aggregando tale contributo su tutti gli alberi della foresta. Questo approccio presenta tuttavia una nota tendenza a favorire variabili con molte modalità (cioè variabili che possono assumere molti valori distinti) ed è applicabile esclusivamente a problemi di classificazione.

Importanza basata sulla Permutazione Considerato l'approccio più robusto e diffuso, questo metodo determina l'importanza di una variabile in modo molto intuitivo: si valuta quanto le prestazioni del modello peggiorano nel momento in cui il legame tra quella variabile e il risultato viene spezzato.

Il processo, eseguito per ogni albero della foresta, si basa sui dati **out-of-bag (OOB)** e funziona così:

1. Si calcola l'accuratezza iniziale dell'albero sui suoi dati OOB.
2. Successivamente, i valori di una sola variabile vengono mescolati casualmente (permutati). In questo modo, la variabile perde ogni sua capacità predittiva.
3. Si ricalcola l'accuratezza dell'albero su questi dati "confusi".

La differenza tra l'accuratezza originale e quella ottenuta dopo la permutazione misura l'importanza della variabile: se l'accuratezza cala drasticamente, significa che il modello si basava molto su quella variabile, che è quindi ritenuta importante. Se, al contrario, l'accuratezza rimane quasi invariata, la variabile è considerata poco influente.

Il punteggio di importanza finale per ogni variabile è dato dalla media di queste differenze, calcolata su tutti gli alberi che compongono la foresta.

Nonostante la loro potenza e versatilità, le Random Forest presentano una criticità fondamentale per il nostro studio. Le implementazioni standard, infatti, non

supportano l'addestramento di modelli su dataset che contengono dati incompleti. Poiché questo lavoro di tesi è incentrato proprio sulla gestione dei *missing data*, questo metodo non verrà impiegato nelle fasi applicative. La loro trattazione è stata tuttavia inclusa per presentare un approccio alternativo e di grande rilevanza nel campo della classificazione, che funge da importante punto di riferimento. [13]

Capitolo 3

Metodologia

3.1 Il dataset

In questo capitolo vengono descritti il dataset, l'approccio sperimentale e gli strumenti di analisi utilizzati per investigare l'efficacia degli split surrogati nella gestione dei dati mancanti. L'obiettivo è validare i concetti teorici esposti precedentemente attraverso un'applicazione pratica su un caso di studio reale e ampiamente riconosciuto.

Per l'analisi è stato selezionato il **dataset UCI Heart Disease**, una raccolta di dati clinici frequentemente utilizzata per la valutazione di algoritmi di classificazione in ambito medico. L'archivio originale della UCI contiene in realtà diversi file separati provenienti da cinque fonti diverse: Cleveland, Ungheria, Svizzera, Long Beach VA e il set di dati Stalog (Heart) e sono dati grezzi e non puliti. Per questo studio si è invece utilizzata una versione del dataset già pulita e pre-elaborata. [14] Mentre il file originale è composto da 76 features, il nostro considera solo 11 attributi clinici e una variabile target.

Di seguito vengono elencate e descritte le principali colonne del dataset:

- `age`: Età del paziente in anni.
- `sex`: Sesso del paziente (1 = maschio, 0 = femmina).
- `chest_pain_type`: Tipologia di dolore toracico. Comprende quattro modalità, che indicano diversi tipi di dolore toracico associato a problemi cardiaci. L'angina tipica corrisponde al dolore classico dovuto a ridotto flusso di sangue al cuore, spesso scatenato da sforzo fisico e alleviato dal riposo. L'angina atipica invece è meno caratteristica, con intensità e localizzazione variabili e

non necessariamente legata all'attività fisica. Il dolore non anginoso deriva da cause non cardiache, come problemi muscoloscheletrici o digestivi, mentre la modalità asintomatica indica l'assenza di dolore percepito, pur potendo esserci comunque un problema cardiaco rilevabile tramite test clinici.

- `resting_bp_s`: Pressione sanguigna a riposo misurata in millimetri di mercurio (mm Hg).
- `cholesterol`: Colesterolo sierico misurato in mg/dl. Livelli più alti possono indicare rischio cardiovascolare maggiore.
- `fasting_blood_sugar`: Glicemia a digiuno > 120 mg/dl (1 = vero, 0 = falso). Un valore alto può essere associato a diabete.
- `resting_ecg`: Indica cosa mostra l'elettrocardiogramma a riposo. Segnala eventuali anomalie nell'attività elettrica del cuore, come cambiamenti in una parte del tracciato chiamata tratto ST-T, che può indicare problemi di afflusso di sangue al cuore, oppure un ispessimento delle pareti dei ventricoli, che significa che il cuore sta lavorando più del normale.
- `max_heart_rate`: Frequenza cardiaca massima raggiunta dal paziente durante uno sforzo fisico.
- `exercise_angina`: Presenza di angina indotta da esercizio (1 = sì, 0 = no). L'angina da sforzo è dolore o fastidio al petto causato da attività fisica.
- `oldpeak`: Depressione del segmento ST indotta dall'esercizio rispetto al riposo. Questo indica quanto si abbassa il segmento ST durante lo sforzo, un segno di ischemia (ridotto flusso di sangue al cuore).
- `st_slope`: Pendenza del segmento ST al picco dell'esercizio. È categorica (up = in salita, flat = piatta, down = in discesa) e riflette la risposta elettrica al stress.
- `target`: Diagnosi di malattia cardiaca (1 = presente, 0 = assente).

In totale, comprende 1190 righe e, analizzando il dataset utilizzato, la distribuzione tra pazienti sani e malati risulta abbastanza bilanciata, con 499 individui senza malattia (target = 0) e 526 con malattia (target = 1).

3.1.1 Strumenti utilizzati

Per condurre gli esperimenti e verificare la reale efficacia degli split surrogati nel nostro specifico dataset, è stato costruito un codice in Python con l'utilizzo di librerie

standard quali pandas e numpy per la manipolazione dei dati, ma anche pacchetti avanzati come "rpart" del linguaggio R, utile per la costruzione di alberi decisionali con surrogate split non supportata direttamente in Python. E' stato quindi utilizzato il pacchetto "rpy2" per eseguire il codice R in Python.

3.1.2 Metodologia sperimentale

Per valutare in modo rigoroso l'impatto degli split surrogati, è stato definito un protocollo sperimentale che simula scenari realistici in cui i dati clinici sono incompleti. La procedura si articola nelle seguenti fasi:

Definizione della Baseline

Come punto di partenza, il modello di classificazione (un albero decisionale basato su algoritmo CART) viene addestrato e valutato sul dataset originale, privo di valori mancanti. Il dataset viene prima suddiviso in un Training Set (70% dei dati) e un Test Set (30%). Il modello viene addestrato sul Training Set e le sue performance vengono misurate sul Test Set. Questo risultato costituisce la **baseline**, ovvero la **massima performance ottenibile dal modello** in condizioni ideali.

Introduzione controllata dei dati mancanti

Successivamente, per simulare scenari con dati incompleti, vengono generate diverse versioni del dataset originale. Su ciascuna versione viene introdotta una percentuale crescente di valori mancanti (ad esempio, 10%, 20% e 30%). I valori vengono rimossi secondo un meccanismo MCAR (Missing Completely At Random), per garantire che l'assenza di un dato sia un evento puramente casuale e non dipendente da altre variabili.

Addestramento e valutazione comparativa

Per ogni dataset con una data percentuale di dati mancanti, viene ripetuta la procedura di divisione in Training e Test Set. Successivamente, vengono addestrati e valutati due modelli distinti sul medesimo set di dati:

- **Con surrogati**: Un albero decisionale in cui la gestione dei dati mancanti è affidata agli split surrogati.
- **Senza surrogati**: Un albero decisionale in cui gli split surrogati sono disattivati. In questo caso, quando il valore necessario a uno split è mancante, l'osservazione viene fatta passare nel ramo più frequente del nodo.

La valutazione finale di entrambi i modelli viene eseguita sul corrispondente Test Set, che non è mai stato utilizzato durante la fase di addestramento, garantendo una stima onesta delle performance.

Metriche di Valutazione

Il confronto quantitativo tra i due scenari si basa su tre metriche chiave, calcolate a partire dalla matrice di confusione:

- **Accuratezza** La proporzione di predizioni corrette. (VP + VN) / Totale
- **Sensibilità** La capacità del modello di identificare correttamente i pazienti malati (veri positivi). VP / (VP + FN)
- **Specificità** La capacità del modello di identificare correttamente i pazienti sani (veri negativi). VN / (VN + FP)

La scelta di queste tre metriche non è casuale. Mentre l'accuratezza fornisce una misura generale della performance, in un contesto di diagnosi medica è fondamentale analizzare gli errori in modo più specifico. La sensibilità è cruciale per minimizzare il rischio di mancate diagnosi (falsi negativi), mentre la specificità è importante per evitare di sottoporre persone sane a trattamenti non necessari (falsi positivi).

L'intero processo sperimentale è stato automatizzato tramite uno script in Python; il codice sorgente completo è riportato in Appendice A.1, mentre lo script per la generazione dei grafici è disponibile in Appendice A.2.

Capitolo 4

Risultati

4.1 Risultati

In questa sezione si espongono i risultati ottenuti mediante il procedimento metodologico illustrato nel capitolo precedente.

4.1.1 Definizione della Baseline

La prima fase sperimentale ha avuto come obiettivo la creazione di un modello di riferimento (baseline), addestrato sul dataset originale completo. In questa condizione ideale, l'addestramento di un modello con o senza gli split surrogati produce risultati di performance identici, poiché il meccanismo dei surrogati si attiva solo in presenza di dati mancanti. Per questo motivo, si omette l'uso degli split surrogati. Come primo passo, è stata verificata la completezza dei dati, confermando l'assenza di valori mancanti in tutte le colonne, come riportato in Tabella 4.1.

Quindi è stato addestrato un albero decisionale sul dataset originale completo. La valutazione del modello sul Test Set ha prodotto un'accuratezza complessiva dell'**82.4%**. Le performance dettagliate sono riassunte nella matrice di confusione presentata in Tabella 4.2.

Dalla matrice di confusione sono state calcolate le metriche di performance chiave per il modello di baseline:

- **Accuratezza**: $\frac{169+125}{169+125+42+21} = \frac{294}{357} \approx 82.4\%$
- **Sensibilità**: $\frac{169}{169+21} = \frac{169}{190} \approx 88.9\%$
- **Specificità**: $\frac{125}{125+42} = \frac{125}{167} \approx 74.9\%$

Tabella 4.1: Tabella per il conteggio di dati mancanti nel dataset.

Variabile	**Conteggio Valori Mancanti**
`age`	0
`sex`	0
`chest_pain_type`	0
`resting_bp_s`	0
`cholesterol`	0
`fasting_blood_sugar`	0
`resting_ecg`	0
`max_heart_rate`	0
`exercise_angina`	0
`oldpeak`	0
`st_slope`	0
`target`	0

Tabella 4.2: Matrice di confusione del modello di baseline sul Test Set.

	Predetto Sano (0)	**Predetto Malato (1)**
Reale Sano (0)	125 (VN)	42 (FP)
Reale Malato (1)	21 (FN)	169 (VP)

Questi valori rappresentano lo standard con cui verranno confrontate le performance dei modelli addestrati su dataset con dati mancanti.

Per comprendere il funzionamento interno del modello, è stata analizzata la sua struttura, visualizzata in Figura 4.1.

L'analisi visuale dell'albero evidenzia il ruolo centrale della variabile `st_slope`, utilizzata come primo criterio di suddivisione nel nodo radice. Per confermare quantitativamente questa osservazione, è stata calcolata l'importanza di ciascuna variabile, come mostrato in Tabella 4.3.

Tabella 4.3: Importanza delle variabili per il modello di baseline, calcolata dall'algoritmo `rpart`. Un valore più alto indica un maggiore potere predittivo della variabile.

Variabile	**Punteggio di Importanza**
`st_slope`	145.89
`chest_pain_type`	44.96
`sex`	10.54
`oldpeak`	9.23
`resting_bp_s`	4.99
`exercise_angina`	3.96

Dall'analisi dell'albero decisionale si evince che `st_slope` è la feature con il maggiore

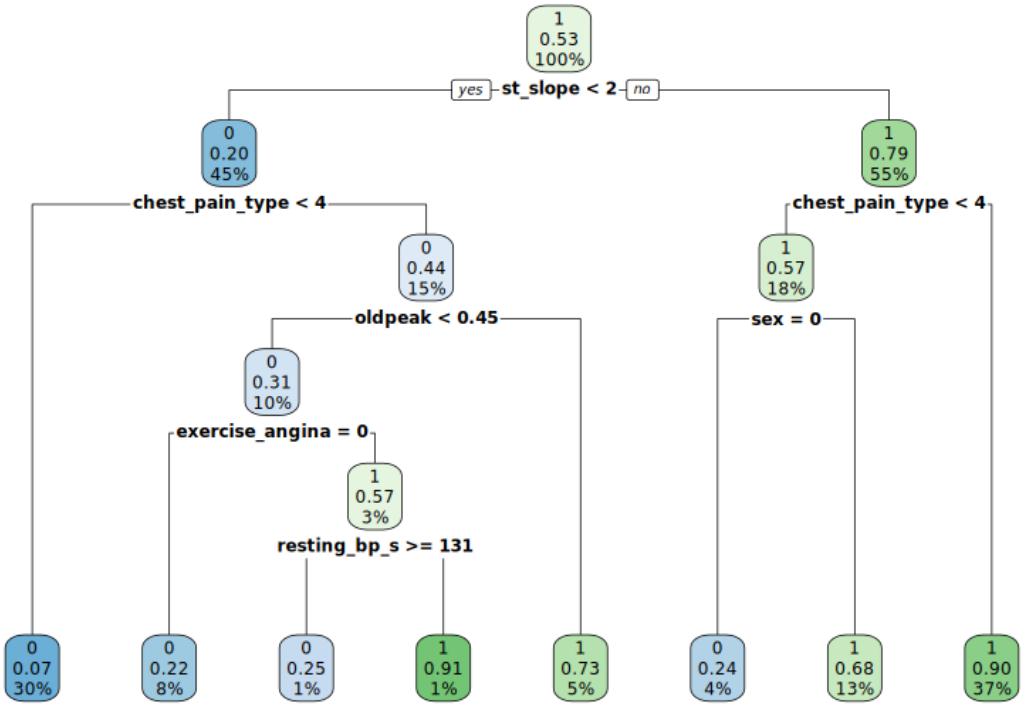


Figura 4.1: Struttura dell'albero decisionale addestrato sul dataset completo. I nodi in blu rappresentano una previsione di assenza di malattia, mentre quelli in verde indicano una previsione di presenza di malattia. Ogni nodo riporta la classe predetta, la probabilità della classe vincente e la percentuale di osservazioni che ricadono in quel nodo.

potere discriminante. La regola iniziale `st_slope < 2` suddivide infatti i pazienti in due rami principali: se la condizione è soddisfatta, il paziente ha una probabilità significativamente più alta di essere sano; in caso contrario, viene indirizzato verso un percorso ad alto rischio di malattia. Successivamente, il modello utilizza criteri diversi per i due rami. Nel ramo sinistro, la variabile `chest_pain_type` diventa decisiva: un valore inferiore a 4 conduce a una classificazione di "sano" con elevata probabilità. Nel ramo destro, invece, la variabile successiva è il sex, indicando che il sesso del paziente è un fattore rilevante in questo specifico ramo. Proseguendo nell'analisi, si osserva come l'albero utilizzi altre variabili come oldpeak solo in percorsi specifici. È interessante notare come alcune feature appaiano solo nel ramo destro o sinistro, a dimostrazione della capacità del modello di scoprire complesse relazioni tra le variabili, che un'analisi più semplice potrebbe non rilevare.

4.1.2 Introduzione controllata dei dati mancanti

Una volta stabilito il valore di accuratezza massimo che questo modello ha da offrire (**82,4%**), la seconda fase prevede l'introduzione di dati mancanti e il confronto dell'accuratezza con o senza split surrogati.

4.1.3 Introduzione del 10% di valori mancanti. Analisi senza e con gli split surrogati

Il primo scenario sperimentale prevede l'introduzione artificiale del 10% di dati mancanti. Si verifica, inizialmente, che l'algoritmo per l'introduzione dei missing stia funzionando a dovere, analizzando quindi l'output in Tabella 4.4.

Analisi senza split surrogati

Si considera il dataset appena creato omettendo l'uso degli split surrogati. In questo caso, come già detto, quando un valore necessario a uno split risulta mancante, l'osservazione viene instradata nel ramo più frequente del nodo. Il modello, una volta addestrato, ha prodotto un'accuratezza del **77,6%**. La matrice di confusione risultante, riportata in Tabella 4.5, ci ha permesso di calcolare le metriche di performance:

- **Accuratezza**: $\frac{166+111}{166+111+56+24} = \frac{277}{357} \approx 77.6\%$
- **Sensibilità**: $\frac{166}{166+24} = \frac{166}{190} \approx 87.4\%$
- **Specificità**: $\frac{111}{111+56} = \frac{111}{167} \approx 66.5\%$

Tabella 4.4: Conteggio dei valori mancanti per colonna dopo l'introduzione artificiale del 10% di dati incompleti.

Variabile	**Conteggio Valori Mancanti**
`age`	123
`sex`	127
`chest_pain_type`	122
`resting_bp_s`	111
`cholesterol`	107
`fasting_blood_sugar`	97
`resting_ecg`	125
`max_heart_rate`	121
`exercise_angina`	117
`oldpeak`	132
`st_slope`	127
`target`	0

Tabella 4.5: Matrice di confusione per il modello addestrato su dati con il 10% di valori mancanti, senza l'utilizzo di split surrogati.

	Predetto Sano (0)	**Predetto Malato (1)**
Reale Sano (0)	111 (VN)	56 (FP)
Reale Malato (1)	24 (FN)	166 (VP)

Si illustra ora lo schema (Figura 4.2) dell'albero decisionale addestrato dal modello.

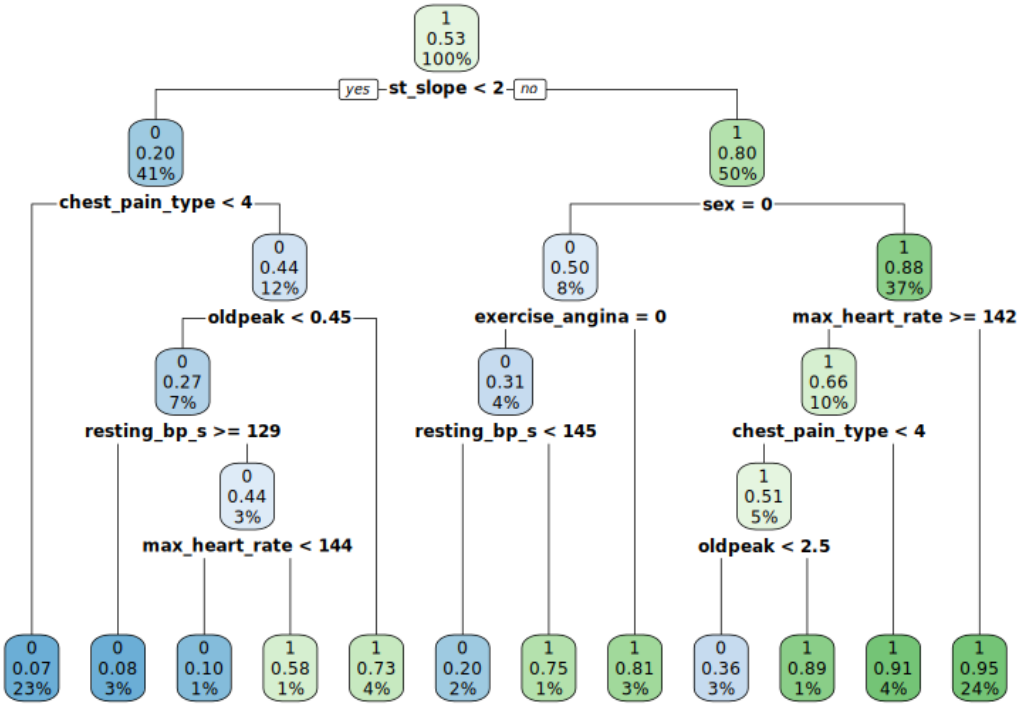


Figura 4.2: Struttura dell'albero decisionale addestrato sul dataset con il 10% di dati mancanti. I nodi in blu rappresentano una previsione di assenza di malattia, mentre quelli in verde indicano una previsione di presenza di malattia. Ogni nodo riporta la classe predetta, la probabilità della classe vincente e la percentuale di osservazioni che ricadono in quel nodo.

In questa configurazione, senza gli split surrogati, la struttura dell'albero è definita esclusivamente dagli split primari. Anche in presenza del 10% di dati mancanti, la variabile `st_slope` si conferma come il predittore più potente, mantenendo il suo ruolo di discriminante principale e continua a separare efficacemente il dataset in un ramo sinistro, associato a una bassa probabilità di malattia (paziente "sano"), e un ramo destro, associato a un'alta probabilità di malattia (paziente "malato"). Analizzando i due rami separatamente, si osserva un comportamento interessante. Nel ramo sinistro (a basso rischio), il modello utilizza la feature `chest_pain_type` come criterio successivo. Questo ramo si sviluppa in un sotto-albero relativamente complesso che sfrutta ulteriori variabili come `oldpeak`, `resting_bp_s` e `max_heart_rate` per classificare con accuratezza i pazienti con bassa probabilità di malattia. Nel ramo destro (ad alto rischio), invece, la struttura è diversa. Dopo la divisione iniziale basata su sex, il modello utilizza criteri differenti per i due sottogruppi. È notevole come per il sottogruppo di pazienti con sex=0 (donne), il modello utilizzi la variabile `exercise_angina` e poi `resting_bp_s`, mentre per gli uomini (sex=1) si

basi su `max_heart_rate` e `chest_pain_type`. Questa differenza strutturale tra i rami e la scelta di variabili diverse per sottogruppi differenti (es. uomini vs. donne nel ramo ad alto rischio) conferma la capacità dell'albero di adattarsi ai dati e di modellare relazioni complesse, anche se la sua performance è peggiorata a causa dei valori mancanti.

Analisi con split surrogati

A questo punto si addestra lo stesso modello sul dataset con il 10% di missing, utilizzando questa volta gli split surrogati.

I risultati mostrano un netto recupero delle performance. L'accuratezza complessiva sale all'**80.7%**, con un guadagno di oltre 3 punti percentuali rispetto allo scenario senza surrogati. Dalla matrice di confusione, in Tabella 4.6, si ottiene:

- **Accuratezza**: $\frac{163+125}{163+125+42+27} = \frac{288}{357} \approx 80.7\%$
- **Sensibilità**: $\frac{163}{163+27} = \frac{163}{190} \approx 85.8\%$
- **Specificità**: $\frac{125}{125+42} = \frac{125}{167} \approx 74.9\%$

Tabella 4.6: Matrice di confusione per il modello addestrato su dati con il 10% di valori mancanti, con l'utilizzo degli split surrogati.

	Predetto Sano (0)	**Predetto Malato (1)**
Reale Sano (0)	125 (VN)	42 (FP)
Reale Malato (1)	27 (FN)	163 (VP)

In questo caso l'albero decisionale di Figura 4.3 mostra una struttura diversa rispetto a quello senza split di Figura 4.2. Infatti in questo caso, mentre lo split radice è pressochè identico, incontriamo differenze sostanziali nella gestione degli split primari dal secondo ordine in poi. La differenza sostanziale però è nell'introduzione del secondo layer di split surrogati.

Surrogati nel nodo `st_slope`

Per illustrare l'efficacia del meccanismo, analizziamo in dettaglio il nodo principale del modello. La regola di divisione più discriminante, scelta come split primario, è `st_slope < 1.5`. L'algoritmo, tuttavia, ha preparato una serie di regole alternative in caso questo dato sia mancante. Come si osserva dall'output del modello in tabella 4.7, il primo e più forte split surrogato è basato sulla variabile `oldpeak`, con la regola `oldpeak < 0.95`. Il suo indice di accordo (agree) pari a 0.775 indica che

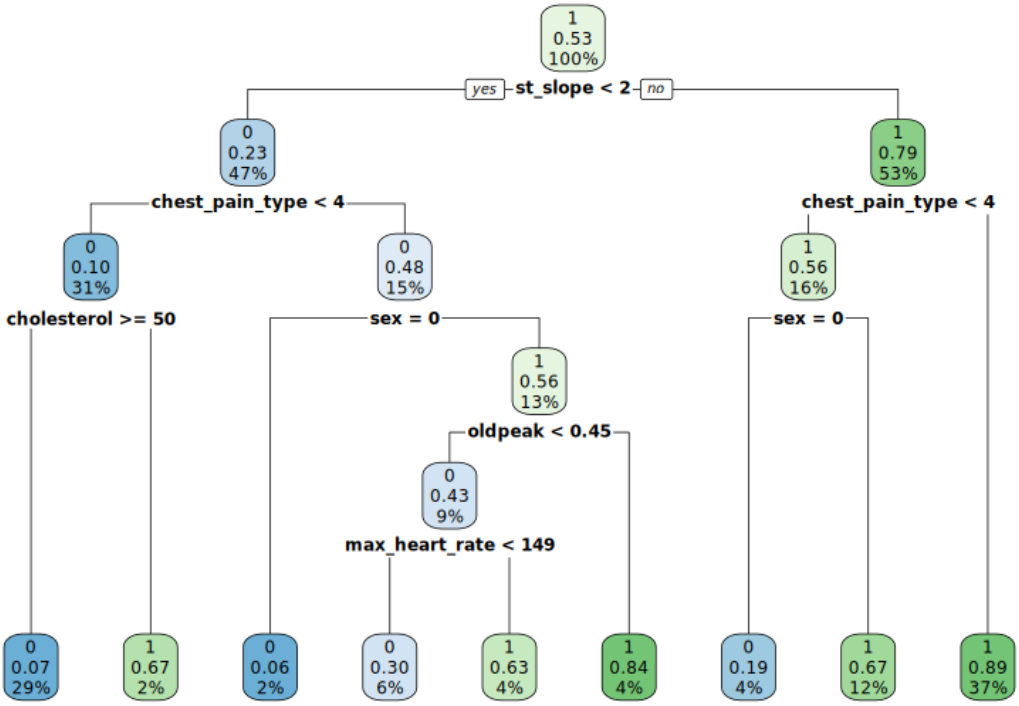


Figura 4.3: Struttura dell'albero decisionale addestrato sul dataset con un 10% di dati mancanti. I nodi in blu rappresentano una previsione di assenza di malattia, mentre quelli in verde indicano una previsione di presenza di malattia. Ogni nodo riporta la classe predetta, la probabilità della classe vincente e la percentuale di osservazioni che ricadono in quel nodo. In questa visualizzazione non sono riportati gli split surrogati, ma solo quelli primari.

questa regola è in grado di replicare la decisione dello split primario nel 77.5% dei casi. Se anche `oldpeak` fosse mancante, il modello passerebbe al secondo surrogato, `exercise_angina < 0.5`, che ha un accordo del 70.3%. Questo processo dimostra come il modello con surrogati non si limiti a ignorare i dati mancanti, ma implementi una strategia gerarchica per utilizzare le informazioni ancora disponibili, preservando così la robustezza della previsione.

Tabella 4.7: Analisi dettagliata dello split per il nodo principale del modello. Vengono mostrati gli split surrogati calcolati per la variabile `st_slope`.

Split Surrogati per `st_slope < 1.5`		
Regola Surrogata	**Direzione**	**Accordo (Agree)**
`oldpeak < 0.95`	a sinistra	0.775
`exercise_angina < 0.5`	a sinistra	0.703
`max_heart_rate < 150.5`	a destra	0.695
`chest_pain_type < 3.5`	a sinistra	0.680
`age < 51.5`	a sinistra	0.624

Questo stesso meccanismo basato su surrogati è stato calcolato dall'algoritmo per ogni altro nodo di divisione dell'albero, garantendo una gestione robusta dei dati mancanti a ogni passo del processo decisionale.

4.1.4 Introduzione del 20% di valori mancanti. Analisi senza e con gli split surrogati

Il secondo scenario sperimentale prevede l'introduzione artificiale del 20% di dati mancanti, processo illustrato in Tabella 4.8.

Analisi senza split surrogati

Analizziamo il caso senza split surrogati. Dalla matrice di confusione, in Tabella 4.6, si ottiene:

- **Accuratezza**: $\frac{162+98}{162+98+69+28} = \frac{260}{357} \approx 72.8\%$
- **Sensibilità**: $\frac{162}{162+28} = \frac{162}{190} \approx 85.3\%$
- **Specificità**: $\frac{98}{98+69} = \frac{98}{167} \approx 58.7\%$

Il modello, una volta addestrato, ha prodotto un'accuratezza del **72,8%**.

Tabella 4.8: Conteggio dei valori mancanti per colonna dopo l'introduzione artificiale del 20% di dati incompleti.

Variabile	**Conteggio Valori Mancanti**
`age`	257
`sex`	245
`chest_pain_type`	259
`resting_bp_s`	209
`cholesterol`	220
`fasting_blood_sugar`	252
`resting_ecg`	234
`max_heart_rate`	225
`exercise_angina`	233
`oldpeak`	236
`st_slope`	248
`target`	0

Tabella 4.9: Matrice di confusione per il modello addestrato su dati con il 20% di valori mancanti, senza l'utilizzo di split surrogati.

	Predetto Sano (0)	**Predetto Malato (1)**
Reale Sano (0)	98 (VN)	69 (FP)
Reale Malato (1)	28 (FN)	162 (VP)

La struttura dell'albero addestrato sul dataset con il 20% di dati mancanti è mostrata in Figura 4.4.

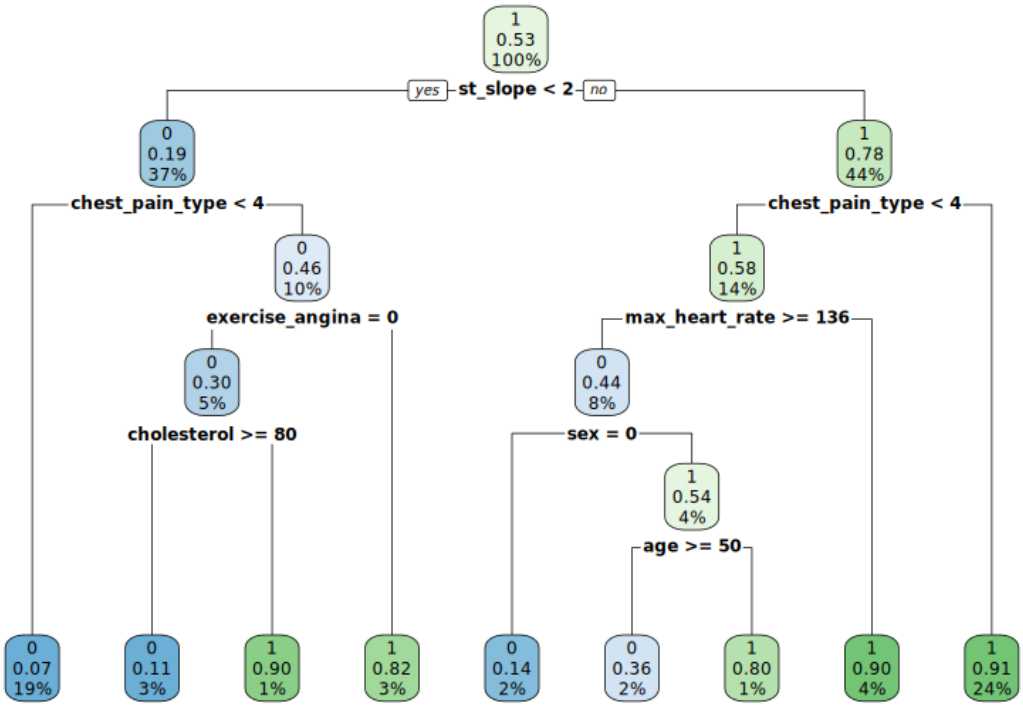


Figura 4.4: Struttura dell'albero decisionale addestrato sul dataset con un 20% di dati mancanti. I nodi in blu rappresentano una previsione di assenza di malattia, mentre quelli in verde indicano una previsione di presenza di malattia. Ogni nodo riporta la classe predetta, la probabilità della classe vincente e la percentuale di osservazioni che ricadono in quel nodo.

Pur mantenendo una certa somiglianza con i modelli precedenti, si possono notare due tendenze significative. In primo luogo, la variabile `st_slope` si conferma ancora una volta come il predittore più robusto, rimanendo il criterio di suddivisione del nodo principale. Tuttavia, la struttura dei rami mostra una crescente instabilità. A differenza dei modelli precedenti, l'algoritmo ora introduce nuove variabili nei rami inferiori (ad esempio, `cholesterol >= 80` e `max_heart_rate >= 136` in posizioni diverse). Inoltre al variare della distribuzione dei missing nel dataset cambia nettamente la struttura dell'albero e le variabili decisionali utilizzate. Questa continua riorganizzazione strutturale è un chiaro indicatore del fatto che il modello sta lottando con un'informazione sempre più povera. La sua incapacità di trovare un percorso decisionale stabile riflette la perdita di accuratezza causata dall'aumento dei valori mancanti.

Analisi con split surrogati

A questo punto si addestra lo stesso modello sul dataset avente il 20% di missing, utilizzando questa volta l'opzione per attivare gli split surrogati.

I risultati mostrano ancora una volta un netto recupero delle performance. L'accuratezza complessiva sale all'**80.9%**, con un guadagno evidente rispetto allo scenario senza surrogati. Dalla matrice di confusione, in Tabella 4.10, si ottiene:

- **Accuratezza**: $\frac{151+138}{151+138+29+39} = \frac{289}{357} \approx 81.0\%$
- **Sensibilità**: $\frac{151}{151+39} = \frac{151}{190} \approx 79.5\%$
- **Specificità**: $\frac{138}{138+29} = \frac{138}{167} \approx 82.6\%$

Tabella 4.10: Matrice di confusione per il modello addestrato su dati con il 20% di valori mancanti, con l'utilizzo degli split surrogati.

	Predetto Sano (0)	**Predetto Malato (1)**
Reale Sano (0)	138 (VN)	29 (FP)
Reale Malato (1)	39 (FN)	151 (VP)

Analizzando la struttura dell'albero addestrato con l'ausilio degli split surrogati (Figura 4.5), si osserva un evidente aumento della sua complessità rispetto ai modelli precedenti.

Surrogati nel nodo `st_slope`

Per illustrare l'efficacia del meccanismo in questo scenario con il 20% di dati mancanti, si analizza nuovamente il comportamento del nodo radice del modello. La regola di divisione più discriminante, scelta come split primario, si conferma essere `st_slope < 1.5`. Come nel caso precedente, l'algoritmo ha preparato una serie di regole alternative per gestire l'eventuale assenza di questo dato. Come si evince dall'output dettagliato (Tabella 4.11), il primo e più forte split surrogato è basato sulla variabile `oldpeak`, con la regola `oldpeak < 0.85`. Il suo indice di accordo (agree) pari a 0.761 indica che questa regola è in grado di replicare la decisione dello split primario nel 76.1% dei casi. Se anche `oldpeak` fosse mancante, il modello passerebbe al secondo surrogato, `max_heart_rate < 148.5`, che ha un accordo del 71.8%. Questo processo dimostra come, anche con un maggior numero di dati mancanti, il modello con surrogati implementi una strategia gerarchica per utilizzare le informazioni ancora disponibili, preservando così la robustezza della previsione. Questo stesso meccanismo, basato sul calcolo dei surrogati, è stato applicato dall'algoritmo

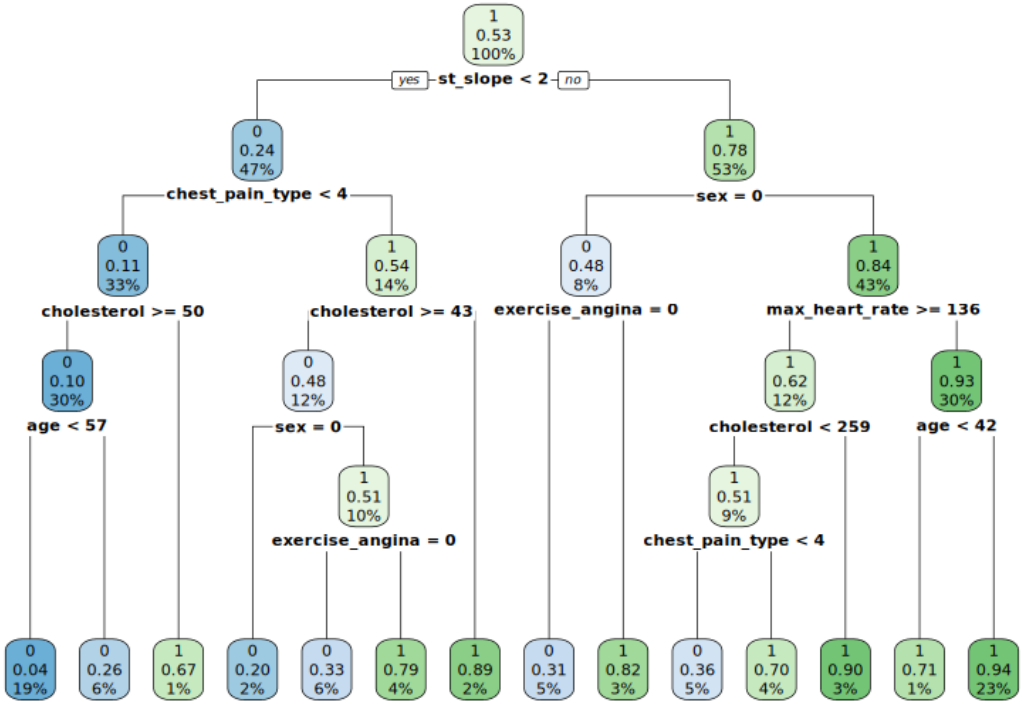


Figura 4.5: Struttura dell'albero decisionale addestrato sul dataset con un 20% di dati mancanti con gli split surrogati. I nodi in blu rappresentano una previsione di assenza di malattia, mentre quelli in verde indicano una previsione di presenza di malattia. Ogni nodo riporta la classe predetta, la probabilità della classe vincente e la percentuale di osservazioni che ricadono in quel nodo.

Tabella 4.11: Analisi dettagliata degli split surrogati per il nodo radice del modello, addestrato con il 20% di dati mancanti.

Split Surrogati per `st_slope < 1.5`		
Regola Surrogata	**Direzione**	**Accordo (Agree)**
`oldpeak < 0.85`	a sinistra	0.761
`max_heart_rate < 148.5`	a destra	0.718
`exercise_angina < 0.5`	a sinistra	0.687
`chest_pain_type < 3.5`	a sinistra	0.672
`age < 51.5`	a sinistra	0.652

a ogni altro nodo di divisione dell'albero, garantendo una gestione robusta dei dati mancanti lungo l'intero processo decisionale.

4.1.5 Confronto complessivo e analisi del degrado delle performance

Per fornire una visione d'insieme dell'impatto dei dati mancanti e quantificare l'efficacia degli split surrogati, i risultati di tutti gli scenari sperimentali sono stati unificati e confrontati. La Tabella 4.12 riassume le principali metriche di performance (Accuratezza, Sensibilità e Specificità) al variare della percentuale di dati mancanti.

Tabella 4.12: Riepilogo comparativo delle performance dei modelli al variare della percentuale di dati mancanti. Tutti i valori sono espressi in percentuale (%).

	Senza Split Surrogati			**Con Split Surrogati**		
% Missing	**Acc.**	**Sens.**	**Spec.**	**Acc.**	**Sens.**	**Spec.**
0%	82,4	88,9	74,9	82,4	88,9	74,9
10%	77,6	87,4	66,5	80,7	85,8	74,9
20%	72,8	85,3	58,7	81,0	79,5	82,6
30%	68,4	83,2	51,5	77,9	78,9	76,6
40%	66,1	86,8	42,5	73,1	73,7	72,5
50%	65,0	89,5	37,1	75,4	81,6	68,3
60%	60,5	88,9	28,1	71,1	78,9	62,3
70%	57,7	95,3	15,0	69,2	76,8	60,5
80%	58,5	97,4	14,4	63,9	73,7	52,7
90%	56,3	97,9	9,0	57,1	82,1	28,7

È importante sottolineare che i dati appena presentati derivano da una singola esecuzione del processo sperimentale. La procedura di introduzione dei dati mancanti, essendo del tutto casuale, introduce una fonte di **incertezza**. Rieseguendo gli esperimenti con una diversa distribuzione casuale di 'NaN' nel dataset, i valori delle metriche di performance subiscono leggere oscillazioni.

Per quantificare rigorosamente questa incertezza è necessario un approccio basato su simulazioni multiple, ripetendo ogni esperimento più volte per calcolare la performance media e la sua deviazione standard. Questo permetterebbe di associare a ogni risultato un intervallo di confidenza. La mia ipotesi è che questa incertezza sia dominata da come i NaN colpiscono in maggior misura le variabili più importanti (come `st_slope`). Per dimostrarlo, un futuro esperimento dovrebbe isolare questo effetto inserendo i missing in una variabile alla volta. Tuttavia, dato che l'obiettivo

primario di questa tesi è dimostrare il vantaggio offerto dagli split surrogati, si è ritenuto sufficiente condurre l'analisi su un singolo esperimento controllato per ogni scenario. Al solo scopo di verificare che vi fosse, comunque, sostanziale coerenza tra run diversi, ho eseguito il codice più volte constatando che i risultati erano essenzialmente in accordo, con un vantaggio di alcuni punti percentuali nelle figure di merito tra i run con e senza surrogate split. Un'analisi approfondita dell'incertezza viene lasciata come possibile e interessante sviluppo futuro di questo lavoro.

Di seguito si riportano i grafici di confronto tra le diverse metriche nei casi con e senza split surrogati.

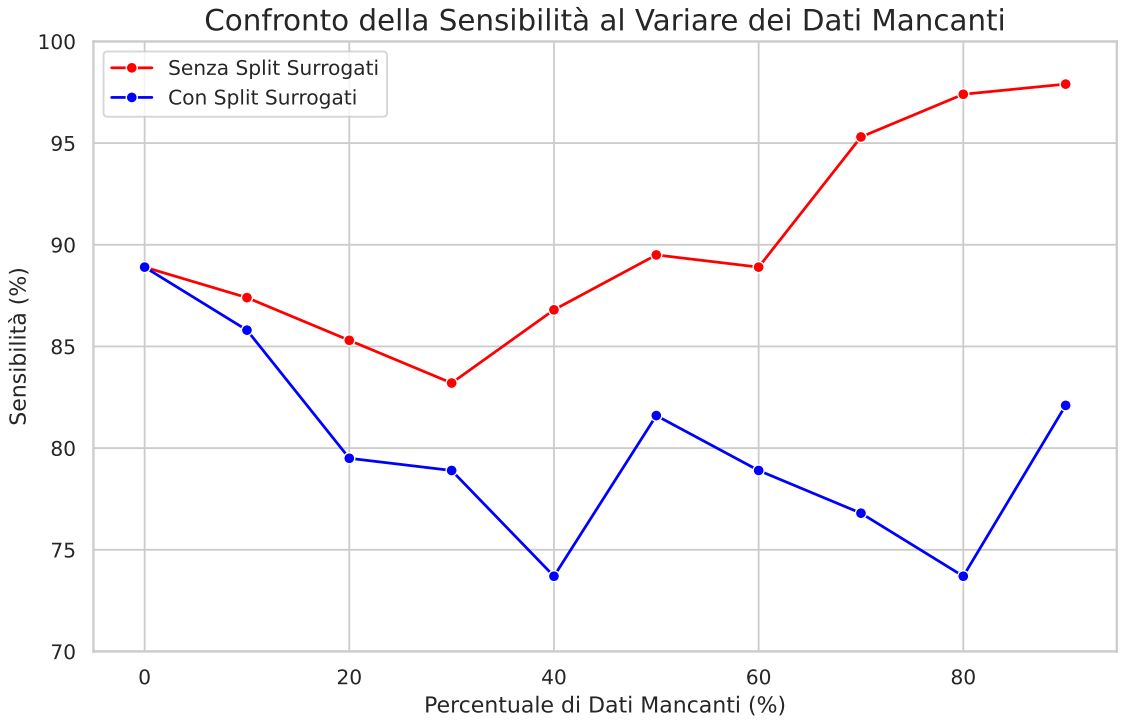


Figura 4.6: Grafico di confronto della sensibilità al variare della percentuale di dati mancanti.

L'analisi comparativa evidenzia la superiore robustezza del modello che impiega gli split surrogati. Come illustrato nel grafico dell'accuratezza (Figura 4.8), la performance del modello senza surrogati subisce un crollo rapido e costante, perdendo oltre 25 punti percentuali nello scenario con il 90% di dati mancanti. Al contrario, la curva di performance del modello con surrogati mostra una degradazione molto più contenuta e controllata, mantenendo un vantaggio significativo in tutti gli scenari di dati incompleti.

Ancora più rivelatrice è l'analisi della sensibilità (Figura 4.6) e della specificità (Figura 4.7). Il modello senza surrogati, nel tentativo di compensare la perdita di

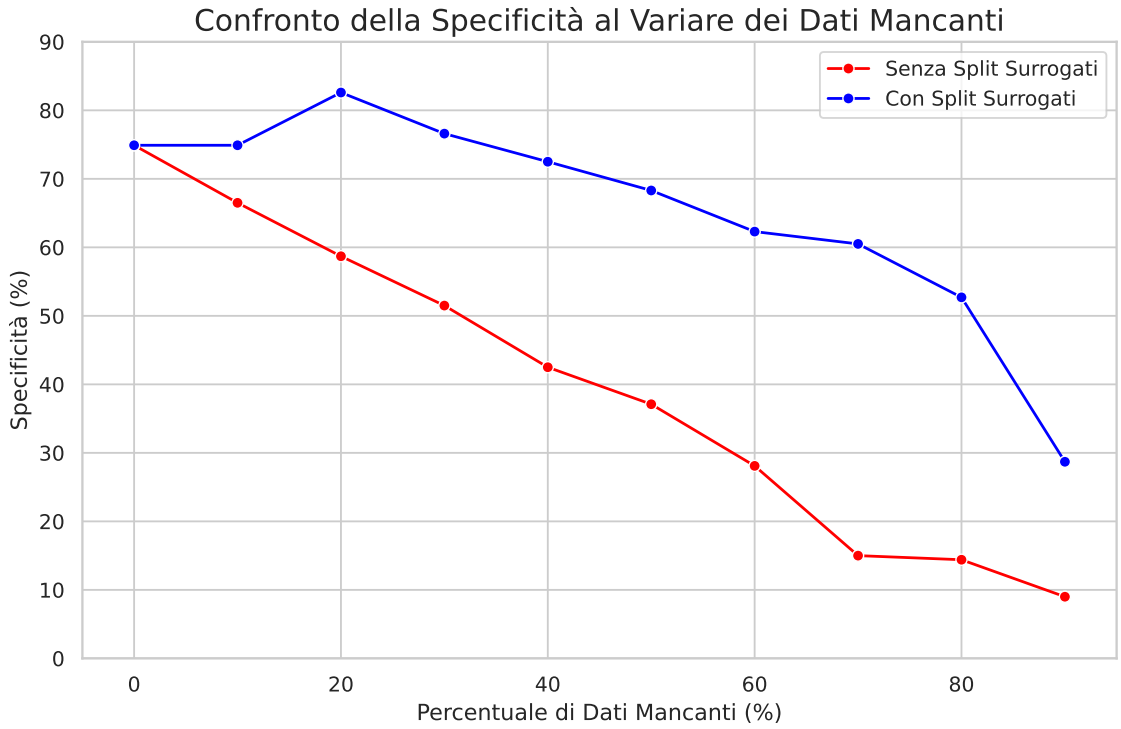


Figura 4.7: Grafico di confronto della specificità al variare della percentuale di dati mancanti.

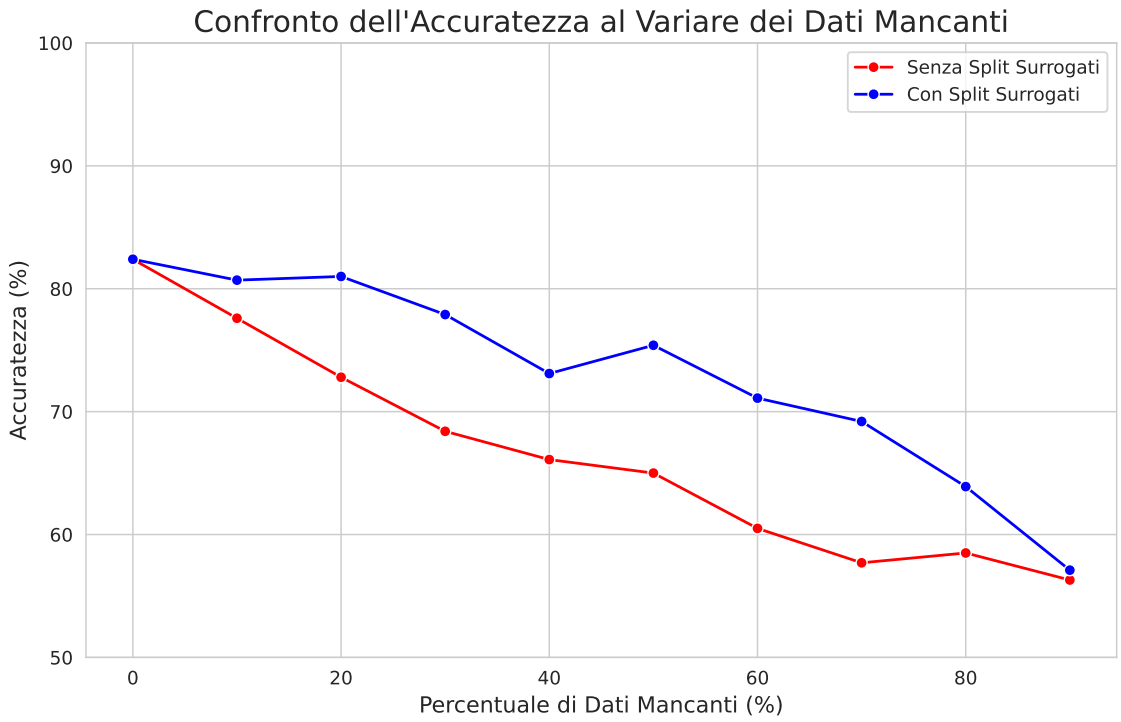


Figura 4.8: Grafico di confronto dell'accuratezza al variare della percentuale di dati mancanti.

informazione, collassa verso una previsione predefinita: la sua sensibilità aumenta fino a quasi il 98%, a fronte di un crollo netto della specificità, che scende al di sotto del 10%. Questo comportamento descrive un modello inutilizzabile, che classifica quasi ogni paziente come "malato" pur di non sbagliare una diagnosi.

Il modello con split surrogati, invece, dimostra di essere notevolmente più equilibrato. Pur con un inevitabile calo delle performance a percentuali di missing estreme, riesce a mantenere sia la sensibilità che la specificità su valori relativamente alti e bilanciati (superiori al 50% fino al 70-80% di dati mancanti). Questo indica che il meccanismo dei surrogati non solo migliora l'accuratezza, ma, cosa forse ancora più importante, previene il collasso del modello, preservandone la capacità di discriminare tra le due classi in modo affidabile anche in condizioni di forte incertezza.

Infine, all'osservazione dei grafici in Figura 4.8 si nota che all'aumentare della percentuale di dati mancanti l'accuratezza del modello senza split surrogati decresce più rapidamente rispetto al modello con split surrogati. Una stima visiva suggerisce che il degrado del modello privo di una strategia per i dati mancanti avvenga a un ritmo circa 2–3 volte superiore rispetto al modello che utilizza gli split surrogati. Sebbene questo valore sia indicativo, conferma che l'impiego degli split surrogati aumenta significativamente la robustezza del modello.

Capitolo 5

Conclusioni

In questa tesi si è stato affrontato il problema dei dati mancanti nelle applicazioni di Machine Learning per la medicina, con un focus sulla diagnosi di patologie cardiache.

L'obiettivo era valutare l'efficacia degli split surrogati, una tecnica intrinseca ai modelli ad albero, come strategia per gestire l'incompletezza dei dati. A tal fine, è stato condotto uno studio sperimentale sul dataset UCI Heart Disease, confrontando le performance di un albero decisionale standard con quelle di un modello che implementa gli split surrogati, in scenari con percentuali crescenti di dati mancanti (10%, 20%, 30%, ecc.) introdotti artificialmente.

I risultati hanno dimostrato che il **modello standard** subisce un significativo **degrado** delle performance in presenza di dati incompleti. Al contrario, il **modello con split surrogati** si è rivelato notevolmente più **robusto**, riuscendo a mantenere un'accuratezza significativamente più elevata e, soprattutto, a preservare un equilibrio tra sensibilità e specificità. Ciò conferma che gli split surrogati costituiscono una strategia efficace per mitigare l'impatto negativo dei dati mancanti, garantendo previsioni più affidabili.

Questo studio presenta alcuni limiti, tra cui l'utilizzo di un singolo dataset, la generazione di dati mancanti secondo un meccanismo puramente casuale (MCAR) e l'esecuzione di una singola simulazione per ogni scenario. Di conseguenza, gli sviluppi futuri potrebbero includere la validazione di questi risultati su altri dataset clinici e un confronto diretto delle performance con quelle ottenute tramite metodi di imputazione avanzata come **MICE**. Sarebbe inoltre opportuna la conduzione di **esperimenti multipli**, così da ottenere una stima più robusta delle performance. Infine, risulterebbe interessante estendere lo studio ai modelli **Random Forest**, che, come già detto, si basa sull'utilizzo simultaneo di più alberi decisionali.

Bibliografia

[1] I. T. Jolliffe, *Principal Component Analysis*, 2nd edition, Springer, New York, 2002.

[2] R. A. Fisher, *The use of multiple measurements in taxonomic problems*, *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.

[3] K. Fukushima, *Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position*, *Biological Cybernetics*, vol. 36, no. 4, pp. 193–202, 1980.

[4] G. De Nunzio, *Machine Learning: principi di funzionamento e applicazioni in Medicina*, *Ithaca: Viaggio nella Scienza XIX*, 2022.

[5] M. Rivolta, *Metodi di imputazioni per dati mancanti: applicazione al dataset INVALSI*, Tesi di Laurea Magistrale, Politecnico di Milano, 2015.

[6] *The PEAS Project, National Centre for Research Methods (NCRM), "Using imputation for missing values," Practical Exemplars and Survey Analysis online resource*, disponibile online: `http://www.restore.ac.uk/PEAS/imputation.php`. (Consultato in data: 15-12-2025).

[7] G. Bernardo, *Analisi e ricostruzione dei profili di carico di utenze elettriche*, Tesi di Laurea Magistrale, Politecnico di Torino, 2021.

[8] L. Venuto, *Strategie di Imputazione in Ambiti Complessi: Studio Comparativo tra Modelli MICE e modelli di Machine Learning*, Tesi di Laurea, Università degli studi di Padova, 2023.

[9] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, Cambridge, MA, 2016. Disponibile online: `https://www.deeplearningbook.org`, vedi Capitolo 14: *Autoencoders*. (Consultato in data: 21-12-2025).

[10] J. Li, W. Ren, M. Han, *Variational auto-encoders based on the shift correction for imputation of specific missing in multivariate time series*, Measurement,

vol. 186, 2021, Art. no. 110055, Elsevier. DOI: https://doi.org/10.1016/j.measurement.2021.110055. (Consultato in data: 22-12-2025).

[11] I. Goodfellow, Y. Bengio, A. Courville, *Chapter 10. Sequence Modeling: Recurrent and Recursive Nets*, in *Deep Learning*, MIT Press, 2016, disponibile online: http://www.deeplearningbook.org. (Consultato in data: 22-12-2025).

[12] L. Breiman, J. Friedman, R. Olshen, C. Stone, *Classification and Regression Trees*, Chapman & Hall, 1984.

[13] A. Hapfelmeier, *Analysis of Missing Data with Random Forests*, Tesi di dottorato, Ludwig-Maximilians-Universität München, 2012.

[14] R. Huijzer, *UCI Heart Disease Dataset (Processed and Combined)*, GitHub repository, 2023. Disponibile online: https://github.com/rikhuijzer/heart-disease-dataset (Consultato in data: 27-12-2025).

Appendice A

Codice Sorgente

Questa appendice contiene il codice sorgente principale utilizzato per l'analisi e la modellazione descritte nella tesi.

A.1 Creazione degli alberi decisionali e analisi delle metriche

Il seguente script Python è quello utilizzato nella metodologia per la creazione dei modelli di alberi decisionali, la modifica controllata della percentuale di valori mancanti nel dataset e la valutazione delle prestazioni tramite le principali metriche di output.

Parametri di configurazione degli esperimenti Nel codice riportato è possibile controllare il livello di incompletezza del dataset e l'utilizzo degli split surrogati tramite pochi parametri configurabili. In particolare, la percentuale di valori mancanti introdotti artificialmente nel dataset può essere modificata agendo sulla variabile `MISSING_PERCENTAGE`, espressa in percentuale rispetto al totale delle osservazioni delle variabili esplicative.

L'attivazione o la disattivazione degli split surrogati negli alberi decisionali è invece controllata dal parametro `usesurrogate`, che assume valore pari a 0 per disabilitare l'uso degli split surrogati e valore pari a 1 (o 2) per abilitarli durante la fase di addestramento e di predizione del modello.

```
"""
The UCI Heart Disease dataset is a widely used dataset in the field of machine learning and data science for predicting heart disease. This dataset is part of the UCI Machine Learning Repository and has been utilized in numerous research studies and projects.
Overview
The dataset contains information about patients, including various medical attributes that are used to predict the presence of heart disease. The dataset is a combination of data from five different sources: Cleveland, Hungarian, Switzerland, Long Beach VA, and Stalog (Heart) Data Set. In total, the dataset comprises 1190 instances with 11 common features.
Features
The dataset includes the following features:•
    age: Age of the patient in years•
    sex: Gender of the patient (1 = male, 0 = female)•
    cp: Chest pain type (4 values)•
    trestbps: Resting blood pressure (in mm Hg on admission to the hospital)•
    chol: Serum cholesterol in mg/dl•
    fbs: Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)•
    restecg: Resting electrocardiographic results (values 0, 1, 2)•
    thalach: Maximum heart rate achieved•
    exang: Exercise-induced angina (1 = yes, 0 = no)•
    oldpeak: ST depression induced by exercise relative to rest•
    slope: The slope of the peak exercise ST segment•
    ca: Number of major vessels (0-3) colored by fluoroscopy•
    thal: Thalassemia (3 = normal, 6 = fixed defect, 7 = reversible defect)
Usage
The dataset is commonly used for training and evaluating machine learning models to predict heart disease. Various classifiers such as Random Forest, SVM, Logistic Regression, KNN, and Decision Tree have been applied to this dataset1.

The UCI Heart Disease dataset is a valuable resource for
```

```
    researchers and practitioners in the field of machine
    learning and healthcare. It provides a comprehensive set of
    features that can be used to develop predictive models for
    heart disease, contributing to better diagnosis and treatment
     planning2.
Altre informazioni:
1.  https://github.com/harinda0/Heart-Disease-Prediction-ML/blob
    /main/Final%20Project.ipynb
2.  https://github.com/rikhuijzer/heart-disease-dataset
- - - - - - - - -
https://gts.ai/dataset-download/uci-heart-disease-data/  il
    dataset
Riporta tutte le informazioni compresa la variabile target:
Num: presence or absence of heart disease

https://archive.ics.uci.edu/dataset/45/heart+disease  il dataset

"""




import pandas as pd
import numpy as np
from rpy2.robjects import pandas2ri, r
from rpy2.robjects.packages import importr
import rpy2.robjects as robjects
from rpy2.robjects.conversion import localconverter

def insert_missing_values_by_percentage(df, target_col,
    missing_percentage):
    """
    Inserts a specified percentage of missing values (NaN)
       randomly into the feature columns of a DataFrame.

    Args:
        df (pd.DataFrame): The original DataFrame.
        target_col (str): The name of the target column to
           exclude from modifications.
```

```
        missing_percentage (float): The percentage of values to replace with NaN (e.g., 5 for 5%).

    Returns:
        pd.DataFrame: A new DataFrame with the specified percentage of missing values.
    """
    # Creare una copia per non modificare il DataFrame originale
    df_missing = df.copy()

    # Separare le feature dal target
    features = df_missing.drop(columns=[target_col])

    # Calcolare il numero totale di elementi nelle feature
    n_rows, n_cols = features.shape
    total_elements = n_rows * n_cols

    # Calcolare il numero di valori da rendere mancanti
    n_missing = int(total_elements * (missing_percentage / 100))

    if n_missing == 0:
        print(f"Attenzione: Con {missing_percentage}%, nessun valore verrà reso mancante. Aumenta la percentuale o la dimensione dei dati.")
        return df_missing

    # Creare una maschera booleana (tutti False all'inizio)
    mask = np.full(features.shape, False)

    # Scegliere 'n_missing' indici piatti (da 0 a total_elements-1) in modo casuale e unico
    flat_indices = np.random.choice(total_elements, n_missing, replace=False)

    # Convertire gli indici piatti in indici (riga, colonna) e impostare True nella maschera
    row_indices, col_indices = np.unravel_index(flat_indices, features.shape)
    mask[row_indices, col_indices] = True

```

```
    # Applicare la maschera al DataFrame delle feature per inserire i NaN
    features_with_nan = features.where(~mask, np.nan)

    # Riunire le feature modificate con la colonna target originale
    df_missing[features.columns] = features_with_nan

    return df_missing



def train_rpart_with_surrogates(df, target_col, test_size=0.3, random_state=42,
                                cp=0.01, minsplit=20, maxdepth=10, usesurrogate=2, maxsurrogate=5):
    """
    Train an rpart decision tree with surrogate splits for missing data

    Parameters:
    - df: pandas DataFrame
    - target_col: string, name of target column
    - test_size: float, proportion for validation set
    - random_state: int, random seed for reproducibility
    - cp: complexity parameter
    - minsplit: minimum number of observations in a node to split
    - maxdepth: maximum depth of the tree
    - usesurrogate: how to use surrogates (0=no, 1=yes, 2=use surrogates for missing data)
    - maxsurrogate: the number of surrogate splits retained in the output

    Returns:
    - trained_model: R rpart model object
    - predictions: predictions on test set
    - accuracy: accuracy on test set
    """
```

```python

    print("----------------------------")
    print("train_rpart_with_surrogates")
    print("----------------------------")
    print("usesurrogate = " + str(usesurrogate))
    print("maxsurrogate = " + str(maxsurrogate) + "\n")

    # Import R packages
    rpart = importr('rpart')
    base = importr('base')

    # Prepare features and target
    X = df.drop(columns=[target_col])
    y = df[target_col]

    # Convert to R objects using context manager and assign to R global environment
    with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
        robjects.globalenv["r_X"] = robjects.conversion.py2rpy(X)
        robjects.globalenv["r_y"] = robjects.conversion.py2rpy(y)


    # Split data in R
    r_code = f"""
    set.seed({random_state})

    # Create combined dataframe
    model_data <- cbind(r_X, target = r_y)
    model_data$target <- as.factor(model_data$target)

    # Split into training and validation
    n <- nrow(model_data)
    train_size <- floor({1-test_size} * n)
    train_indices <- sample(seq_len(n), size = train_size)

    train_data <- model_data[train_indices, ]
    test_data <- model_data[-train_indices, ]
```

```

    # Remove row names to avoid issues
    rownames(train_data) <- NULL
    rownames(test_data) <- NULL
    """
    robjects.r(r_code)

    # Train rpart model with surrogate splits
    # https://www.rdocumentation.org/packages/rpart/versions/4.1.24/topics/rpart.control
    r_code_train = f"""
    # Train rpart model with surrogate splits enabled
    rpart_model <- rpart(
        target ~ .,
        data = train_data,
        method = "class",
        control = rpart.control(
            minsplit = {minsplit},
            minbucket = round({minsplit}/3),
            cp = {cp},
            maxdepth = {maxdepth},
            usesurrogate = {usesurrogate},  # Key parameter for surrogate splits
            maxsurrogate = {maxsurrogate},  # Maximum number of surrogate splits to retain
            xval = 10,                      # Number of cross-validations
            surrogatestyle = 1              # 1=percent correct
        ),
        parms = list(
            split = "gini"                  # Use gini index for splitting
        )
    )

    # Print model summary
    print("RPart Model Summary:")
    print(summary(rpart_model))

    # Make predictions on test set
```

```
    predictions <- predict(rpart_model, test_data, type = "class")

    # Calculate accuracy
    accuracy <- mean(predictions == test_data$target, na.rm = TRUE)

    # Get confusion matrix
    conf_matrix <- table(Predicted = predictions, Actual = test_data$target)

    # Get the number of surrogate splits actually used
    frame <- rpart_model$frame
    uses_surrogate <- sum(frame$ncompete > 0 | frame$nsurrogate > 0)
    """

    robjects.r(r_code_train)

    # Get results back to Python
    rpart_model = robjects.r['rpart_model']
    predictions = robjects.r['predictions']
    accuracy = robjects.r['accuracy'][0]
    conf_matrix = robjects.r['conf_matrix']
    uses_surrogate = robjects.r['uses_surrogate'][0]

    print(f"\nRPart Decision Tree trained successfully!")
    print(f"Uses surrogate splits: {uses_surrogate} nodes have surrogate splits")
    print(f"Validation Accuracy: {accuracy:.4f}")
    print(f"Confusion Matrix:")
    print(conf_matrix)

    return rpart_model, predictions, accuracy



# next function not used
def train_rpart_advanced_surrogates(df, target_col, test_size=0.3, random_state=42,
```

```
                                    surrogate_params=None):
    """
    Advanced version with detailed control over surrogate splits
    """

    print("--------------------------------")
    print("train_rpart_advanced_surrogates")
    print("--------------------------------")

    rpart = importr('rpart')

    # Default surrogate parameters
    if surrogate_params is None:
        surrogate_params = {
            'usesurrogate': 2,
            'maxsurrogate': 5,
            'surrogatestyle': 1
        }

    # Convert entire dataframe to R using context manager
    with robjects.conversion.localconverter(robjects.
       default_converter + pandas2ri.converter):
        robjects.globalenv["r_df"] = robjects.conversion.py2rpy(
           df)

    r_code = f"""
    set.seed({random_state})

    # Prepare data
    model_data <- r_df
    model_data${target_col} <- as.factor(model_data${target_col
       })

    # Split data
    n <- nrow(model_data)
    train_size <- floor({1-test_size} * n)
    train_indices <- sample(seq_len(n), size = train_size)

    train_data <- model_data[train_indices, ]
    test_data <- model_data[-train_indices, ]
```

```
    # Train rpart with detailed surrogate control
    rpart_model <- rpart(
        {target_col} ~ .,
        data = train_data,
        method = "class",
        control = rpart.control(
            minsplit = 20,
            minbucket = 7,
            cp = 0.01,
            maxdepth = 10,
            usesurrogate = {surrogate_params['usesurrogate']},
            maxsurrogate = {surrogate_params['maxsurrogate']},
            surrogatestyle = {surrogate_params['surrogatestyle
                '']},
            xval = 10
        )
    )

    # Make predictions
    predictions <- predict(rpart_model, test_data, type = "class
        ")
    accuracy <- mean(predictions == test_data${target_col}, na.
        rm = TRUE)

    # Get detailed information about surrogate usage
    frame <- rpart_model$frame
    total_nodes <- nrow(frame)
    nodes_with_surrogates <- sum(frame$nsurrogate > 0)
    """

    robjects.r(r_code)

    rpart_model = robjects.r['rpart_model']
    predictions = robjects.r['predictions']
    accuracy = robjects.r['accuracy'][0]
    total_nodes = robjects.r['total_nodes'][0]
    nodes_with_surrogates = robjects.r['nodes_with_surrogates'
        ][0]

```

```python
    print(f"Advanced RPart with surrogate splits trained!")
    print(f"Total nodes: {total_nodes}")
    print(f"Nodes using surrogate splits: {nodes_with_surrogates}")
    print(f"Validation Accuracy: {accuracy:.4f}")

    return rpart_model, predictions, accuracy



def get_surrogate_info(model):
    """Get information about surrogate splits in the model"""

    print("------------------")
    print("get_surrogate_info")
    print("------------------")

    with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
        robjects.globalenv["rpart_model"] = robjects.conversion.py2rpy(model)

    #robjects.r['rpart_model'] = model

    r_code = """
    # Get frame information
    frame_df <- as.data.frame(rpart_model$frame)
    frame_df$node_name <- rownames(rpart_model$frame)

    # Count nodes with surrogate splits
    surrogate_nodes <- frame_df[frame_df$nsurrogate > 0, c("node_name", "ncompete", "nsurrogate")]
    """

    robjects.r(r_code)

    # Convert surrogate_nodes back to pandas DataFrame using context manager
    with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
```

```python
        surrogate_nodes = robjects.conversion.rpy2py(robjects.r['surrogate_nodes'])

    print("Nodes using surrogate splits:")
    if len(surrogate_nodes) > 0:
        print(surrogate_nodes)
    else:
        print("No surrogate splits used in this tree")

    return surrogate_nodes




def make_rpart_predictions_with_surrogates(model, new_data):
    """
    Make predictions using the rpart model with surrogate splits
    """

    print("----------------------------------------")
    print("make_rpart_predictions_with_surrogates")
    print("----------------------------------------")

    # Convert new_data to R using context manager
    with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
        r_new_data = robjects.conversion.py2rpy(new_data)

    with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
        robjects.globalenv["rpart_model"] = robjects.conversion.py2rpy(model)
        robjects.globalenv["new_data"] = robjects.conversion.py2rpy(r_new_data)

    # Surrogate splits will be automatically used for missing values during prediction
    predictions = robjects.r('predict(rpart_model, new_data, type = "class")')

```

```
    # Convert predictions back to Python using context manager
    with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
        python_predictions = robjects.conversion.rpy2py(predictions)

    return python_predictions




def get_variable_importance(model):
    """Get variable importance from the model"""

    print("-----------------------")
    print("get_variable_importance")
    print("-----------------------")

    with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
        robjects.globalenv["rpart_model"] = robjects.conversion.py2rpy(model)
#   robjects.r['rpart_model'] = model

    r_code = """
    # Get variable importance
    var_imp <- rpart_model$variable.importance
    var_imp_df <- data.frame(
        Variable = names(var_imp),
        Importance = as.numeric(var_imp)
    )
    var_imp_df <- var_imp_df[order(-var_imp_df$Importance), ]
    """

    robjects.r(r_code)

    # Convert variable importance back to pandas DataFrame
    with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
        var_imp_df = robjects.conversion.rpy2py(robjects.r['
```

```
            var_imp_df'])

    return var_imp_df

def plot_rpart_tree(model, filename=None):
    """Plot the decision tree and save to file"""

    print("---------------")
    print("plot_rpart_tree")
    print("---------------")

    try:
        rpart_plot = importr('rpart.plot')
        with robjects.conversion.localconverter(robjects.default_converter + pandas2ri.converter):
            robjects.globalenv["rpart_model"] = robjects.conversion.py2rpy(model)

        if filename:
            r_code = f"""
            png('{filename}', width=800, height=600)
            rpart.plot(rpart_model, main="Decision Tree with Surrogate Splits")
            dev.off()
            """
        else:
            r_code = """
            rpart.plot(rpart_model, main="Decision Tree with Surrogate Splits")
            """

        robjects.r(r_code)
        print(f"Tree plot saved to {filename}" if filename else "Tree displayed")

    except Exception as e:
        print(f"Could not plot tree: {e}")
        print("Please install rpart.plot package in R: install.packages('rpart.plot')")

```

```python



# Example usage

print("-----")
print("main")
print("-----")

if __name__ == "__main__":

    datasetnumber = 1  # 0: synthetic data, 1: UCI Heart Disease dataset , 2: Fisher iris

    match datasetnumber:
        case 0:
            # Create sample data with missing values
            np.random.seed(42)
            n_samples = 500

            sample_df = pd.DataFrame({
                'age': np.random.randint(20, 70, n_samples),
                'income': np.random.normal(50000, 20000, n_samples),
                'education': np.random.randint(1, 5, n_samples),
                'credit_score': np.random.normal(650, 100, n_samples),
                'target': np.random.randint(0, 2, n_samples)
            })

            # Introduce missing values
            missing_income = np.random.random(n_samples) < 0.15
            missing_credit = np.random.random(n_samples) < 0.15
            sample_df.loc[missing_income, 'income'] = np.nan
            sample_df.loc[missing_credit, 'credit_score'] = np.nan

        case 1:   # use the UCI Heart Disease dataset
        # Load the dataset
            url = "https://raw.githubusercontent.com/rikhuijzer/
```

```
                heart-disease-dataset/main/heart-disease-dataset.
                csv"
            sample_df = pd.read_csv(url)

            # --- 2. INTRODUCIAMO I MISSING SU TUTTO IL DATASET
                ---
            MISSING_PERCENTAGE = 90.0
            print("="*60)
            print(f"FASE 1: Introduzione del {MISSING_PERCENTAGE
                }% di valori mancanti sull'intero dataset...")
            print("="*60)

            if MISSING_PERCENTAGE > 0:
                df_with_missing =
                    insert_missing_values_by_percentage(
                df=sample_df,
                target_col='target',
                missing_percentage=MISSING_PERCENTAGE
            )
            else:
                df_with_missing = sample_df.copy()

            sample_df = df_with_missing
            # ----------------


            n_samples = len(sample_df)  # sample_df.shape[0]

            # Display the first few rows of the dataset
            print(sample_df.head())

            # Preprocess the dataset (e.g., handling missing
                values, encoding categorical variables)
            # Example: Fill missing values with the mean of the
                column
            #sample_df.fillna(data.mean(), inplace=True)

        case 2:   # Fisher's iris
            from sklearn.datasets import load_iris

```

```
            # Load the Iris dataset
            iris = load_iris()

            # Convert to Pandas DataFrame
            sample_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

            # Add the target column
            sample_df['target'] = iris.target  # [0, 1, 2] correspond to ['setosa', 'versicolor', 'virginica']

            # Display the first few rows
            print(sample_df.head())

            # Choose one of the three species as positive, the other two as negative
            positiveclass = 0
            y = sample_df['target']  # view!!
            y[y == positiveclass] = 99
            y[y != 99] = 0  # negative
            y[y == 99] = 1  # positive
            print(sample_df)

        case _:
            print("DATASET NOT AVAILABLE")
            exit()

    ## Example: Encode categorical variables
    #sample_df['sex'] = data['sex'].map({0: 'female', 1: 'male'})

    print("DataFrame shape:", sample_df.shape)
    print("Missing values per column:")
    print(sample_df.isnull().sum())

    # Train rpart with or without surrogate splits

    usesurrogate = 1 # 0=no, 1=yes, 2=use surrogates for missing data (0 or 2)
```

```
    if usesurrogate == 0:
        maxsurrogate = 0
    else:
        maxsurrogate = 5

    rpart_model, predictions, accuracy = train_rpart_with_surrogates(
        df=sample_df,
        target_col='target',
        test_size=0.3,
        random_state=42,
        cp=0.01,
        minsplit=20,
        maxdepth=5,
        usesurrogate=usesurrogate,
        maxsurrogate=maxsurrogate
    )

    # Get surrogate information
    surrogate_info = get_surrogate_info(rpart_model)

    # Get variable importance
    var_importance = get_variable_importance(rpart_model)
    print("\nVariable Importance:")
    print(var_importance)

    '''
    # Make predictions on new data with missing values
    new_data = sample_df.drop('target', axis=1).iloc[:5].copy()
    new_data.loc[0, 'income'] = np.nan
    new_data.loc[1, 'credit_score'] = np.nan

    print("\nNew data with missing values:")
    print(new_data[['age', 'income', 'credit_score']])

    new_predictions = make_rpart_predictions_with_surrogates(rpart_model, new_data)
    print("Predictions on new data (using surrogate splits):")
    print(new_predictions)
    '''
```

```

    # Plot the tree
    plot_rpart_tree(rpart_model, "decision_tree.png") # save to file




    '''
    R INSTALLATION

https://cran.mirror.garr.it/CRAN/
Scelgo Ubuntu

# update indices
sudo apt update -qq
# install two helper packages we need
sudo apt install --no-install-recommends software-properties-common dirmngr
# add the signing key (by Michael Rutter) for these repos
# To verify key, run gpg --show-keys /etc/apt/trusted.gpg.d/cran_ubuntu_key.asc
# Fingerprint: E298A3A825C0D65DFD57CBB651716619E084DAB9
wget -qO- https://cloud.r-project.org/bin/linux/ubuntu/marutter_pubkey.asc | sudo tee -a /etc/apt/trusted.gpg.d/cran_ubuntu_key.asc
# add the repo from CRAN -- lsb_release adjusts to 'noble' or 'jammy' or ... as needed
sudo add-apt-repository "deb https://cloud.r-project.org/bin/linux/ubuntu $(lsb_release -cs)-cran40/"
# install R itself
sudo apt install --no-install-recommends r-base



Nella console, chiamata da shell con il comando: R

install.packages("rpart")
install.packages('rpart.plot')
```

```

q()


conda create -p ./missing_env python pandas rpy2 scikit-learn


    '''
```

Listing A.1: Script per la creazione degli alberi decisionali, gestione dei valori mancanti e calcolo delle metriche

A.2 Generazione dei grafici riassuntivi delle metriche

Il seguente script è dedicato esclusivamente alla generazione dei grafici complessivi delle metriche di valutazione ottenute dai diversi esperimenti descritti nella metodologia.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# --- DATI DELL'ESPERIMENTO ---

data = {
    '% Missing':        [0, 10,  20,  30,  40,  50,  60,  70,
       80,  90],
    'Acc_Senza':        [82.4, 77.6, 72.8, 68.4, 66.1, 65.0,
       60.5, 57.7, 58.5, 56.3],
    'Sens_Senza':       [88.9, 87.4, 85.3, 83.2, 86.8, 89.5,
       88.9, 95.3, 97.4, 97.9],
    'Spec_Senza':       [74.9, 66.5, 58.7, 51.5, 42.5, 37.1,
       28.1, 15.0, 14.4, 9.0],
    'Acc_Con':          [82.4, 80.7, 81.0, 77.9, 73.1, 75.4,
       71.1, 69.2, 63.9, 57.1],
    'Sens_Con':         [88.9, 85.8, 79.5, 78.9, 73.7, 81.6,
       78.9, 76.8, 73.7, 82.1],
    'Spec_Con':         [74.9, 74.9, 82.6, 76.6, 72.5, 68.3,
       62.3, 60.5, 52.7, 28.7]
}
df = pd.DataFrame(data)

# Imposta stile grafico
sns.set_theme(style="whitegrid")

# --- GRAFICO 1: ACCURATEZZA ---

print("Generazione grafico Accuratezza...")
plt.figure(figsize=(10, 6))
```

```
sns.lineplot(x='% Missing', y='Acc_Senza', data=df, marker='o', label='Senza Split Surrogati', color='red')
sns.lineplot(x='% Missing', y='Acc_Con', data=df, marker='o', label='Con Split Surrogati', color='blue')

# Personalizzazione
plt.title("Confronto dell'Accuratezza al Variare dei Dati Mancanti", fontsize=16)
plt.xlabel("Percentuale di Dati Mancanti (%)", fontsize=12)
plt.ylabel("Accuratezza (%)", fontsize=12)
plt.ylim(50, 90) # Limiti dell'asse Y per una migliore visualizzazione
plt.xlim(-5, 95) # Aggiunge un po' di margine all'asse X
plt.legend()

# Salva il file
plt.savefig("grafico_accuratezza.pdf", format='pdf', bbox_inches='tight')
print("...salvato in 'grafico_accuratezza.pdf'")
plt.close() # Chiude la figura per liberare memoria

# --- GRAFICO 2: SENSIBILITÀ ---

print("Generazione grafico Sensibilità...")
plt.figure(figsize=(10, 6))
sns.lineplot(x='% Missing', y='Sens_Senza', data=df, marker='o', label='Senza Split Surrogati', color='red')
sns.lineplot(x='% Missing', y='Sens_Con', data=df, marker='o', label='Con Split Surrogati', color='blue')

# Personalizzazione
plt.title("Confronto della Sensibilità al Variare dei Dati Mancanti", fontsize=16)
plt.xlabel("Percentuale di Dati Mancanti (%)", fontsize=12)
plt.ylabel("Sensibilità (%)", fontsize=12)
plt.ylim(70, 100)
plt.xlim(-5, 95)
plt.legend()

# Salva il file
```

```
plt.savefig("grafico_sensibilita.pdf", format='pdf', bbox_inches='tight')
print("...salvato in 'grafico_sensibilita.pdf'")
plt.close()

# --- GRAFICO 3: SPECIFICITÀ ---

print("Generazione grafico Specificità...")
plt.figure(figsize=(10, 6))
sns.lineplot(x='% Missing', y='Spec_Senza', data=df, marker='o', label='Senza Split Surrogati', color='red')
sns.lineplot(x='% Missing', y='Spec_Con', data=df, marker='o', label='Con Split Surrogati', color='blue')

# Personalizzazione
plt.title("Confronto della Specificità al Variare dei Dati Mancanti", fontsize=16)
plt.xlabel("Percentuale di Dati Mancanti (%)", fontsize=12)
plt.ylabel("Specificità (%)", fontsize=12)
plt.ylim(0, 90)
plt.xlim(-5, 95)
plt.legend()

# Salva il file
plt.savefig("grafico_specificita.pdf", format='pdf', bbox_inches='tight')
print("...salvato in 'grafico_specificita.pdf'")
plt.close()

print("\nOperazione completata. Tutti i grafici sono stati generati.")
```

Listing A.2: Script per la generazione dei grafici complessivi delle metriche